



Full Length Article

ARLHNIDS-IoT: An accurate and robust lightweight hybrid-NIDS for IoT network security

Arpita Srivastava, Ditipriya Sinha *

Department of Computer Science & Engineering, National Institute of Technology Patna, Patna Bihar, India

ARTICLE INFO

Keywords:

Hybrid-NIDS
IoT network security
Double deep Q networks
Grey wolf optimization
OPTUNA

ABSTRACT

The rapid growth of IoT networks has heightened vulnerabilities in IoT devices, making them targets for sophisticated attackers. These vulnerabilities compromise sensitive user data and disrupt services for legitimate users. Therefore, securing IoT devices is a crucial research area to protect against cyber-attacks and fully realize the benefits of IoT technologies. Various solutions have already been proposed, including firewalls, antivirus software, and intrusion detection systems (IDS). Firewalls and antivirus software provide only some level of security by filtering known threats and blocking unauthorized access; they primarily rely on predefined rules and filter the packet according to rules without monitoring its behavior in the network. The firewall functions in either inline mode or transparent mode at the network boundary. In contrast, IDS offers more comprehensive protection by continuously monitoring network traffic and system activities, employing both signature-based and anomaly-based detection techniques to identify previously unseen threats (or zero-day attacks). It typically functions in monitoring mode rather than inline with the traffic flow. However, designing an effective IDS for IoT environments presents significant challenges. The limited computational resources of IoT devices make it difficult to process high-dimensional IDS datasets efficiently. On the other hand, dataset imbalance is another major hurdle (as it hinders the accurate identification of intrusive activities, leading to biased detection performance) in the design of efficient intrusion detection systems. To address these challenges, this paper proposes a novel hybrid network intrusion detection system that integrates advanced oversampling, feature selection techniques, and intelligent detection models to enhance the attack detection accuracy and reduce the processing time of intrusion detection in IoT networks. Firstly, the imbalanced IDS dataset is balanced with the help of the modified generative adversarial minority oversampling (GAMO) and fuzzy C-means clustering techniques. In the proposed framework, the existing GAMO model is modified by integrating the attention mechanism to focus on significant patterns in the network traffic and achieve efficacious performance. Hyperparameters of the fuzzy C-means clustering algorithm are optimized using the OPTUNA technique. Secondly, feature selection is performed using the modified Grey Wolf Optimization (GWO) technique by integrating the correlation coefficient in the initialization stage and introducing a novel objective function. This modified feature selection approach reduces the resource constraint and is compatible with IoT networks. Signature-based IDS (SIDS) is designed using the majority voting classifier to detect known attacks. The voting classifier ensembles seven tree-based models, which are optimized using the OPTUNA technique. On the other hand, anomaly-based IDS (AIDS) is proposed, which utilizes the double deep Q-network for unknown DoS attack detection. The proposed “ARLHNIDS-IoT” model integrates SIDS and AIDS in such a way that the incoming IoT-based network traffic is first analyzed by SIDS, which compares it to a database of known attack signatures. When a match is found, the attack is promptly classified, facilitating rapid detection of known threats. If SIDS fails to recognize an attack, the traffic is forwarded to AIDS for further assessment. AIDS inspects any deviations from normal behavior to identify new attacks. The proposed strategy enhances accuracy, minimizes false alarms, and bolsters IoT security. The effectiveness of the proposed “ARLHNIDS-IoT” model is evaluated using three IoT-based datasets: UNSW-NB15, BoT-IoT, and CICIoT2023. Experimental findings from the UNSW-NB15, BoT-IoT, and CICIoT2023 datasets indicate that the proposed method lessens resource limitations by decreasing prediction time by 5.917, 13.523, and 37.751 s, respectively. Additionally, it tackles the imbalanced data challenge by improving recall by 6.06 %, 8.12 %, and 3.86 % while enhancing the F1-score by 5.69 %, 7.95 %, and 2.65 % on UNSW-NB15, BoT-IoT, and

* Corresponding author.

E-mail addresses: arpitas.ph21.cs@nitp.ac.in (A. Srivastava), ditipriyasinha87@gmail.com (D. Sinha).<https://doi.org/10.1016/j.cose.2025.104515>

Received 3 December 2024; Received in revised form 16 April 2025; Accepted 27 April 2025

Available online 29 April 2025

0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

CICIoT2023 datasets, respectively. The accuracy of the SIDS model is observed as 92.89 %, 99.19 %, and 98.31 % on UNSW-NB15, BoT-IoT, and CICIoT2023 datasets, respectively, and the accuracy of the AIDS model is observed as 89.56 %, 90.92 %, and 94.74 % on UNSW-NB15, BoT-IoT, and CICIoT2023 datasets, respectively. The proposed model is evaluated against recent state-of-the-art techniques, demonstrating that the “ARLHNIDS-IoT” model surpasses other contemporary methods.

1. Introduction

The Internet of Things (IoT) expansion has led to a network of interconnected devices that people rely on for daily tasks. These IoT-based environments include smart homes, schools, agriculture, health-care, wearables, transportation, cities, industries, vehicles, and shopping malls. The IoT is expected to bring revolutionary advancements globally. However, several challenges exist in deploying the IoT framework, including social and technical difficulties (Haroon et al., 2016). There are various technical challenges possible within the IoT framework as follows: security, hardware (including power & storage constraints and general hardware technological protocols), standards or policies, gateway systems (including routing protocol, network, and address & sensing issues), middleware (including software & algorithm issue), and IoT databases. The main constraints with IoT devices are limited resources and security. IoT devices are highly susceptible to cyber-attacks, so ensuring their security is paramount. Cyberattacks are increasing worldwide daily at an exponential rate, underscoring the urgent need for robust security measures. IoT devices have encountered various security vulnerabilities and popular cyber-attacks within the IoT-based framework. Denial of Service (DoS), Distributed Denial of Service (DDoS), worms, information manipulation, unauthorized access, and information disclosure (Rahman et al., 2024) are some examples. Cyber attackers are highly skilled at their craft, making it difficult for traditional security measures such as firewalls and antivirus software to detect them accurately. Securing and preventing IoT networks from known and unknown cyber-attacks is extremely challenging. Intrusion detection systems (IDSs) are a key tool in this endeavor. An intrusion detection system (IDS) is a system or software that initially monitors the incoming and outgoing network traffic, generates an alarm if any suspicious activity is detected within the network, and alerts the network administrator (Srivastava and Sinha, 2024).

1.1. The need for accurate and robust NIDS for IoT network security from cyber-attacks

Due to the security challenges and high vulnerability of IoT devices, cyber-attackers can intentionally attack the IoT network and severely harm an individual or a complete network (Alghanam et al., 2023). Therefore, security in the IoT network is necessary. The biggest challenge in designing IDS for IoT network security is that the IoT-based network traffic dataset is highly imbalanced, with many features and target attack categories (Papamartzivanos et al., 2018; Srivastava et al., 2023). This leads to difficulty in accurately detecting malicious traffic as well as the type of malicious behavior. Several state-of-the-art methods have been proposed to overcome the imbalanced data challenge (Rahman et al., 2024; Srivastava et al., 2023; Kumar and Sinha, 2023), using random oversampling, SMOTE, ADASYN, conditional GAN, Wasserstein GAN, and Wasserstein Condition GAN with gradient penalty. On the other hand, many features in the IoT network traffic cause the IDS model to be heavily weighted (Z. Wang et al., 2024; Srivastava and Sinha, 2024; Azimjonov and Kim, 2024), leading to the danger of high resource utilization. Many state-of-the-art methods have already been introduced to overcome the challenges of the large number of features in the IDS domain (Alghanam et al., 2023; Z. Wang et al., 2024; Srivastava and Sinha, 2024; Nazir and Khan, 2021; Chohra et al., 2022; Chowdhury et al., 2023), using nature-inspired-based different metaheuristics (such as genetic algorithm, particle swarm optimization algorithm, ant colony

optimization algorithm, etc.) and non-metaheuristic (such as either filter-based or wrapper-based) techniques. However, most have high complexity or many hyperparameters to tune or do not consider the correlated or essential features. The third challenge is that the SIDS model does not detect new attack traffic data, while the AIDS model detects new attacks but frequently flags false alarms.

To address the above loopholes of the IDS model, this paper designs a lightweight Hybrid Network Intrusion Detection System (Hybrid NIDS) that combines SIDS and AIDS on balanced and effective features in the dataset. It tackles the aforementioned devastating challenges of the IoT network framework by placing the proposed hybrid NIDS in two positions, the IoT gateway and the IoT devices, to secure the IoT network completely. Here, the proposed model describes a six-tiered architecture that leverages the strengths of supervised, unsupervised, and reinforcement learning. It balances the dataset, reduces the dimensionality of the data, manages the detection of known attacks, and ultimately detects unknown denial-of-service (DoS) attacks. The proposed multi-tiered architecture is coined the term “ARLHNIDS-IoT”: An Accurate and Robust Lightweight Hybrid-NIDS for IoT Network Security. The proposed model accurately and robustly detects cyber-attacks on the IoT network. This architecture is also proficient in generating synthetically minority attack data, clustering the majority data samples, extracting the informative and uncorrelated features from the resultant data, intelligently detecting known and unknown attacks, and optimizing the model for optimal results. Since the multi-tiered architecture intelligently detects known and unknown cyber-attacks, it overcomes the security issue of the IoT networks. On the other hand, it reduces the dimensionality of the resultant data by leveraging the benefits of the nature-inspired algorithm; here, it is called the modified GWO-based feature selection technique (discussed in Section 3.6); it overcomes the resource constraint issue of the IoT networks. This paper demonstrates the consistency and effectiveness of the proposed framework by validating it on three IoT-enabled datasets and deploying it in the IoT framework. The proposed “ARLHNIDS-IoT” architecture’s performance is measured in terms of “attack detection accuracy”, “precision”, “recall”, “f1-score”, “false alarm rate”, and “prediction time”. Fig. 1 outlines the block diagram of the proposed model.

1.2. The key contributions of the proposed approach

The main contributions of this paper are outlined as follows:

- (i) To remove the imbalanced dataset challenges in the IDS, this paper modifies the Generative Adversarial Minority Oversampling (GAMO) Technique. This technique alters the original GAMO model by adding an attention mechanism at each neural network layer. The minority class data is provided as input to this module, and the modified GAMO module enhances the number of samples of minority classes to balance the dataset.
- (ii) Since many samples are in the majority classes, the fuzzy C-means clustering approach is used to cluster these samples to get the required number of adequate samples.
- (iii) In the IDS paradigm, feature selection using the exhaustive search method takes $O(2^n)$ and is considered to be an NP-hard problem (Srivastava and Sinha, 2024; Nazir and Khan, 2021), where “n” is the number of features in the dataset. Therefore, a novel nature-inspired approach is adopted for selecting essential

features, which integrates the correlation metric with the modified grey wolf optimization technique.

- (iv) Since no deep neural network model is better than a tree-based classifier (Zoppi et al., 2024), in this paper, for the signature-based intrusion (known attack) detection in the IoT network, an ensemble of seven different tree-based classifiers (such as Decision Tree, Random Forest, Extra Tree, AdaBoost, LightGBM, CatBoost, and Xgboost) is designed utilizing majority voting technique.
- (v) In anomaly-based intrusion (unknown attack) detection in the IoT network, a reinforcement learning-based double deep Q-Network (DDQN) is applied.

1.3. Paper organization

The rest of the paper is organized as follows: Section 2 depicts the existing state-of-the-art techniques, summarizes by analyzing the research gap, and further shows the prime research objective of the proposed method. Section 3 describes the proposed framework in detail by discussing the methodology. Section 4 outlines and discusses the implementation details and experimental results. Last but not least, Section 5 concludes the paper by discussing the drawbacks of the proposed method and future directions to extend this research.

2. Related work

This section discusses some of the state-of-the-art techniques in the IDS domains. Based on the detection techniques used, it is divided into three subsections.

2.1. Related work based on signature-based IDS

Simon et al. (Simon et al., 2022) have proposed a hybrid IDS for securing wireless IoT networks. Here, the hybridization of deep learning and machine learning models is performed. The features in the dataset are extracted using the CNN+RF architecture. The fully connected layer of the CNN module is replaced with the random forest classifier. Zhang & Ma (Zhang and Ma, 2022) have proposed a hybrid IDS model that secures in-vehicle networks by combining rule-based and DNN-based systems in two stages. To reduce the time consumption in the second

stage, the first stage uses the rule-based system to filter the inappropriate rules and select the significant rules. Subsequently, these rules are given to the DNN model in the second stage for classifying malicious and benign traffic. Based on the traffic behavior, an alert signal will be generated. Alghanam et al. (Alghanam et al., 2023) have proposed the NIDS model for the security of IoT networks. Firstly, the dataset is preprocessed by applying the null and duplicate removal, feature normalization, and label encoding of the target class, with zero indicating normal and one indicating the attack samples in the data. Subsequently, the training data is passed to the data clustering phase, where the k-means clustering approach is applied to the normal data from the training dataset. Now, the processed training dataset is given to the tabu local search-based pigeon-inspired optimization model for the feature selection, and the optimal feature subset is produced corresponding to three classifiers such as isolation forest, one-class SVM, and local outlier factor. Now, classification is performed using these classifiers on the preprocessed testing dataset. Each classifier result is an ensemble using the voting classifier, and the performance of the NIDS model is measured. Sharma et al. (Sharma et al., 2024) have designed IDS to secure the IoT networks. Initially, the dataset is preprocessed, which includes data cleaning, normalization, and label encoding. Again, different traditional feature selection methods are checked, and finally, a filter-based feature selection method is adopted to select the essential features from the dataset. Furthermore, at the attack detection or prediction time, two deep learning models, DNN and CNN, are used to make the appropriate prediction among different attack and normal classes. Finally, explainable AI concepts, such as Lime (Local) and SHAP (local and global), are utilized for the model explanation.

Wang et al. (Z. Wang et al., 2024) designed a lightweight IDS model, “BT-TPF,” for the security of IoT networks. Siamese network is utilized to reduce the dimensionality of the data. The new knowledge distillation model is designed to be called the Bert-of-Theseus, where the predecessor contains the encoder part of the vision transformer. In contrast, the successor contains the encoder part of the poolformer. Nandanwar & Katarya (Nandanwar and Katarya, 2024) have proposed a robust deep learning-based intrusion detection system and devised the term “AttackNet” to secure Industrial IoT networks from botnet attacks. Data augmentation, label encoding, and normalization operations are performed in preprocessing. This paper combines the convolutional neural network with the gated recurrent unit to detect significant

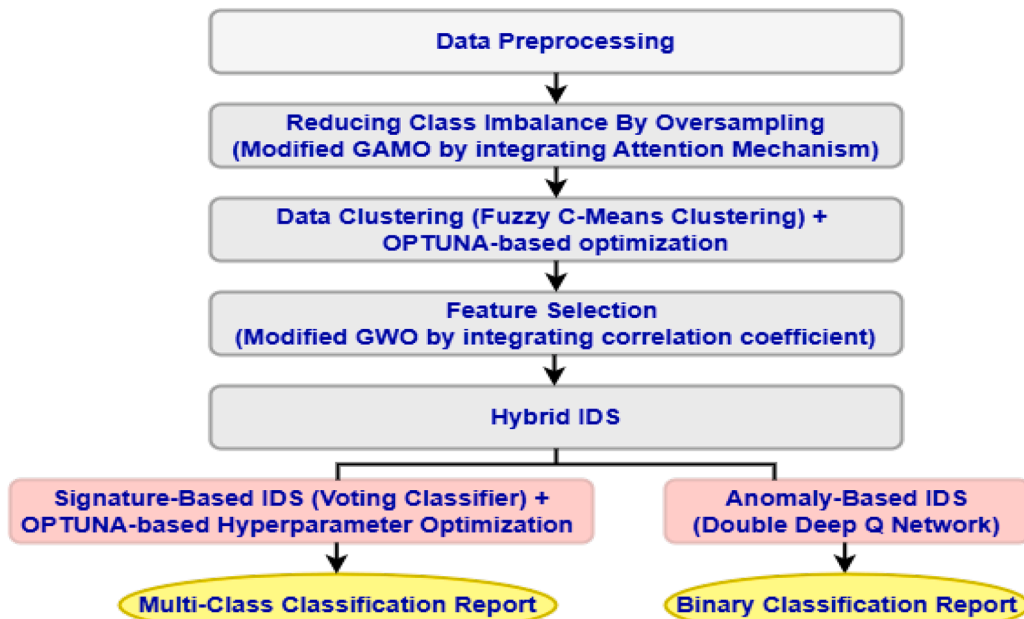


Fig. 1. Block diagram of proposed model.

network-based attacks and classify benign, gafgyt, and mirai. Again, the deep learning model is used for model testing and evaluation. Hizal et al. (Hizal et al., 2024) have designed five distinct IDS models using deep learning techniques to enhance security against DDoS attacks in IoT networks. The CICIoT2023 dataset is utilized, which is further divided into three different datasets based on the attack categories. Pre-processing of the data involves steps like deletion of null records and random selection of the subset of data where the number of features is reduced from 47 to 37. Subsequently, duplicate records are removed, and the data is normalized. Out of five different models, 3 are one-stage models, and the remaining 2 are two-stage models. Some models are used for the binary classification, which classifies into malicious and normal categories; some models are employed for the multi-class classification, which classifies into three categories such as benign, DDoS, and non-DDoS; and finally, the remaining models are utilized for the multi-class classification, which classifies into 12 categories which include ten DDoS attacks subcategories, one non-DDoS attack, and remaining one benign category. Three deep learning models are used for classification: DNN, CNN, and LSTM. The main limitation is that it does not address the imbalanced class problem in the dataset.

2.2. Related work based on anomaly-based IDS

Nguyen & Le (Nguyen and Le, 2023) have proposed an anomaly detection model to detect unknown DoS/DDoS attacks in IoT networks using the fusion of supervised and unsupervised approaches. Initially, feature standardization and feature extraction are performed. The *soft ordering convolutional neural network* model classifies malicious and benign traffic during attack detection. The outputted benign traffic is given to the *local outlier factor* model, which further filters the abnormal behavior from the normal traffic. Again, the outputted normal traffic is given to the *isolation-based anomaly detection using the nearest neighbor ensembles* model, which further classifies abnormal and normal traffic. This time, the normal traffic is passed within the network, and all the abnormal traffic is sent to the administrator. Wu et al. (Wu et al., 2024) have proposed the idea of active learning to design an IDS based on DQN to detect zero-day attacks. The model automatically selects the most valuable data samples for labeling, avoiding the time and cost of manual labeling and model retraining. In addition, a flexible threshold setting improves model stability. The model performs better than other deep learning models on the two given datasets, NSL-KDD and UNSW_NB15; however, the real-time and adaptability of the system have not been evaluated, and the calculation is more complex. Srivastava and Sinha (Srivastava and Sinha, 2025) have proposed an unknown DoS/DDoS attack detection model using the statistical method. Here, the model is divided into two main categories: signature extraction phase and unknown attack detection phase. In the first phase of the model, the signatures are extracted using the FP-Growth algorithm, while in the detection phase, Jaccard similarity is applied to check the similarity score. The similarity between the new packets and the stored attack signatures in the high-volume attack knowledge base is computed. The experiment is conducted on two data sets: real data and traditional CICIDS2017 datasets. Shen et al. (Shen et al., 2024) have proposed an intrusion detection system that detects zero-day attacks in the social Internet of things (SIoT) architecture utilizing the deep q-network. Here, new samples of the SIoT data are created using sample creator; subsequently, samples are selected. Each selected sample is given as input to the classifier and examiner center for the observation. These obtained observations are provided as input to the DQN (designed using the LSTM network) for heuristic learning, which further works as a classification model to identify between attack and benign samples.

Cevallos M. et al. (Rizzardi et al., 2024) have proposed “NERO” for zero-day attack detection for IoT-based environments. It consists of three components: processor, encoder, and decoder. The dataset is divided into two categories, type_A and type_B, based on the zero-day attack categories. Initially, IoT-based low-training data is provided as

input to the encoder module (a task-specific neural), which extracts abstract input data with high dimensionality. This high-dimensional data is provided as input to the processor module, which is a message-passing neural system. Now, the output of the processor module is given to the decoder module (a prototypical neural). Here, two decoder modules are utilized, one for identifying known attacks and another for identification of type_A and type_B zero-day attacks. This output is reflected in the IoT environment and iteratively training the same pipeline again. Toure et al. (Touré et al., 2024) have designed an attack detection framework that utilizes supervised and unsupervised learning approaches for identifying known and zero-day attacks, respectively, on real-time traffic data. A boosting technique detects known attacks, which ensembles five learners: 1D-CNN, DT, RF, KNN, and NB. An unsupervised learning approach, k-means clustering, is used to cluster the data using real-time data, which clusters into attack, normal, and both. A correlation table shows the correlation between the supervised approach’s target category and the clustering algorithm’s output. Distance is checked; the traffic is declared a zero-day attack if it exceeds the pre-defined threshold. Here, hyperparameters of the boosting technique are not optimized. Yu et al. (Yu et al., 2023) have designed a DC-IDS to secure the Industrial IoT. This approach uses the deep Q-network and Markov decision process for the classification, and a variational autoencoder is utilized within the deep Q-network. It is assumed that the reconstruction loss of the known traffic is less than that of the unknown traffic. Hence, reconstruction loss is employed to identify unknown traffic. Here, the ToN-IoT dataset is used for experimentation purposes.

2.3. Related work based on hybrid IDS

Yang et al. in (Yang et al., 2021) have proposed *MTH-IDS*, a hybrid intrusion detection system for securing intravehicular and external vehicular networks. Four tree-based classifiers, such as decision tree, random forest, extra tree, and xgboost, are stacked and optimized using Bayesian optimization with a tree-structured parzen estimator (BO-TPE) for signature-based intrusion detection. In anomaly-based intrusion detection, the biased classifier uses k-means label clustering with a certain confidence threshold of 0.933 and Bayesian optimization with Gaussian processes (BO-GP) for optimization. Feature selection is performed using well-known techniques: information gain, fast correlation-based filter, and kernel principal component analysis, along with Bayesian optimization with the Gaussian process. The traditional SMOTE technique is used for data oversampling. Li et al. (Li et al., 2024) have proposed HDA-IDS, a hybrid intrusion detection system for securing IoT networks from DoS and botnet attacks. Similar to the above paper, the traditional SMOTE technique is used for data oversampling, information gain along with fast correlation-based filter are used for feature selection, and stacking of multiple tree-based classifiers is used for known attack detection. A new technique for detecting unknown attacks is using the CL-GAN (CNN-LSTM-GAN) model. Optimization of different tree-based classifiers is performed using the Hyperopt TPE technique. Aydin et al. (Aydin et al., 2009) have introduced a hybrid IDS that combines anomaly-based and misuse-based IDS techniques. The anomaly-based IDS combines network traffic and packet header anomaly detection techniques, while the misuse-based IDS utilizes the traditional SNORT-based technique. PHAD and NETAD are added to the misuse-based IDS (SNORT) one by one as a preprocessor, and the attack detection rate increases. SNORT, SNORT+ PHAD, and SNORT+PHAD+NETAD detect 27, 51, and 146 attacks, respectively. Vashishtha et al. (Vashishtha et al., 2023) have designed a hybrid IDS model that combinantly detects known and unknown intrusions within a network. They have created and stored malicious signatures in the database, while unknown attack signatures are also created from unknown malicious data. They have used a rule-based model to detect known attacks. In contrast, they have used a majority voting classifier to detect unknown attacks, which ensembles gradient boosting, random

forest, and neural network models. Kim et al. (Kim et al., 2014) have proposed a hybrid intrusion detection system that combines misuse-based and anomaly-based IDS. The data traffic is initially passed to the misuse-based IDS, which uses the C4.5 decision tree algorithm to classify the traffic as malicious or normal. Subsequently, the normal data traffic is decomposed into multiple data traffic using the decision tree classifier. Multiple 1-class SVM classifiers are used at the anomaly-based IDS phase, each using decomposed data traffic to classify malicious or normal patterns.

Ozkan-Okay et al. (Ozkan-Okay et al., 2021) have introduced a hybrid IDS model that proposes a feature selection and grouping method (FSAP) based on a pre-defined threshold. The misuse-based IDS rules extraction uses essential features of the resultant dataset. In contrast, anomaly-based IDS rules extraction uses content features from the consequent dataset. 17 and 13 significant features are selected on the KDD'99 and UNSW-NB15 datasets, respectively, and binary classification is performed. Kanna & Santhi (Kanna and Santhi, 2022) have proposed a hybrid intrusion detection system devising a term BWO—CONV-LSTM (*black widow optimization with convolutional long short-term memory*) model. In the first stage, this paper performs the preprocessing, which includes transforming the labels, removing the redundancy, and normalizing them. This paper uses the nature-inspired ABC (Artificial Bee Colony) algorithm for the feature selection task in the next stage. In the final stage of this paper, a hybrid classifier called BWO—CONV-LSTM is used on the map-reduce framework for the classification. This classification model combines a convolutional neural network and a long short-term memory network, where the output of the convolutional neural network is fed as input to the long short-term memory network with a BWO-based optimization framework for optimizing the hyperparameters of the CNN and LSTM networks. To extract the spatial and temporal features from the data, deep learning models CNN and LSTM are combined. Alghayadh & Debnath (F. Alghayadh and Debnath, 2020; F. Alghayadh and Debnath, 2020) have proposed a two-tier hybrid intrusion detection system architecture for smart home security. In the first tier, a random forest algorithm is utilized to extract the network behavior of the data traffic. In the second tier, misuse-based detection is performed using the user behavior according to user features. These tiers are arranged sequentially, which means that the request is initially passed to the first tier to extract the network behavior. After processing at the first tier, the request is passed to the second tier to detect anomalies according to the user feature (or behavior). Khraisat et al. (Khraisat et al., 2019) have proposed a hybrid IDS that combines signature-based and anomaly-based IDS approaches. Initially, the preprocessing and feature selection processes are performed. Here, an information gain-based feature selection approach is adopted to select the essential features from the preprocessed dataset. The proposed model is divided into two stages: SIDS (signature-based IDS) and AIDS (anomaly-based IDS). The signature-based IDS uses the C5 decision tree classifier to classify between the malicious and normal traffic samples. Subsequently, the benign traffic samples identified by the signature-based IDS are fed as input to the anomaly-based IDS to train the AIDS model with the normal profile, and any traffic that deviates from the normal behavior is flagged as an alarm. Table 1 shows the comparative analysis of the related work with hybrid IDS techniques.

2.4. Research gap

The variety of devices linked to the IoT network is expanding, and the data traffic they produce is constantly evolving; there is a high research interest in developing models more resilient to new and existing types of attacks (Hizal et al., 2024). IDS datasets are highly imbalanced, meaning that some target categories have many samples called the majority class data. In contrast, the rest have very few samples called minority class data. The results of the model which is highly biased toward the majority target category data and offer a biased result that is not accurate and true. Moreover, IDS datasets have many features,

leading to a heavy-weighted and high-dimensional model. Some research solves the high dimensionality of the model using the exhaustive search, which results in NP-hard problems. Meta-heuristic algorithm-based feature selection (Srivastava and Sinha, 2024; Nazir and Khan, 2021) solves the NP-hard problem by selecting the optimal feature subset iteratively. However, the exhaustive search-based feature selection exponentially increases the time complexity. On the other hand, signature-based IDS only detects the known attack traffic but fails to identify the unknown. Meanwhile, anomaly-based IDS detects unknown attack traffic but frequently flags false alarms. Motivated by these challenges, this paper proposes a model that addresses the challenges posed by imbalanced datasets, resource constraints, and the diverse nature of cyber-attacks targeting IoT devices. Developing lightweight and hybrid NIDS for IoT networks is essential to efficiently and accurately detect malicious traffic and protect IoT devices from known and unknown cyber-attacks.

2.5. Research objective (RO)

This section discusses how the research aims to detect cyber-attacks on IoT networks. The top-tier objective is to analyze the IoT network traffic from the IoT dataset and effectively identify known and unknown cyber-attacks. Tackling the challenges of the IoT network framework, such as network security and limited resource constraints of IoT devices, by designing a novel hybrid NIDS model for network security. This paper efficiently fuses supervised, unsupervised, and reinforcement learning to design a hybrid network intrusion detection system that fosters attack detection performance. This paper has established the following primary research objectives:

- RO-1.** Synthetically, minority attack class data generation whose probability distribution is nearly similar to real attack data from the IoT-based network attack dataset.
- RO-2.** Clustering majority class data, optimizing the number of clusters, and picking the most influential data samples from each cluster.
- RO-3.** Extract valuable, uncorrelated, and quality features from the dataset that maximize classification accuracy while minimizing errors, false alarms, and execution time.
- RO-4.** Designing the SIDS model to classify known attacks and normal data samples and optimize the model. Additionally, the AIDS model is designed to detect unknown attack data samples. Validating the effectiveness and consistency of the proposed novel hybrid network intrusion detection system.

This research aims to address the network security challenge by designing a hybrid NIDS model. The following section gives a detailed description of the proposed framework of this paper.

3. Proposed framework

In this section, a broad description of the proposed framework is considered. The proposed model can be discussed from seven perspectives. Firstly, this paper examines the need for the proposed model by taking a practical use case architecture. Now, the remaining six subsections discuss the proposed model. Initially, preprocessing of the dataset is performed by considering several operations, such as encoding, missing or null value imputation, clamping extreme values, and feature scaling. Subsequently, majority and minority classes in the dataset are identified, and the impact of the data imbalanced issue on the model's performance is scrutinized. Fuzzy C-means clustering is performed on the majority classes of the dataset to make each class's compelling and less complex samples. To achieve the optimized result, the hyperparameters of the fuzzy C-means clustering are optimized using the OPTUNA-based technique. The minority classes are given as input to the generative model, which generates synthetic samples of these classes to balance the dataset. This phase of the proposed model

Table 1
Comparative analysis of related work with Hybrid IDS techniques.

Paper	Type of IDS	Type of Problem	Datasets	Clustering	Oversampling	Feature Selection	Known-Attack Detection	Unknown-Attack Detection	Optimization	Result
Yang et al. (Yang et al., 2021)	Hybrid IDS	Supervised, Unsupervised	CICIDS2017, CAN-intrusion	k-means	SMOTE	Information gain, Fast correlation-based filter, Kernel principal component analysis	DT, RF, ET, and Xgboost	CL-k-means with biased classifier	BO-GP, BO-TPE	A detection rate of 99.80 % on known attacks and 75.94 % on unknown attack detection.
Li et al. (Li et al., 2024)	Hybrid IDS	Supervised, Unsupervised	NSL-KDD, BoT-IoT, CICIDS2018	k-means	SMOTE	Information gain, fast correlation-based filter	AdaBoost, Xgboost, RF, ET, DT	GAN, CNN, LSTM	Hyperopt TPE	An accuracy of 99.92 % on known attacks and 98.53 % on unknown attack detection.
Aydin et al. (Aydin et al., 2009)	Hybrid IDS	Supervised	IDEVAL	–	–	–	SNORT	PHAD and NETAD	–	The hybrid approach detects more attacks than the single approach.
Vashishtha et al. (Vashishtha et al., 2023)	Hybrid IDS	Supervised	NSL-KDD, CICIDS2017, UNSW-NB15	–	–	Information gain evaluation algorithm	Rule-based approach	Majority voting of gradient boosting, random forest, and neural network classifiers	–	The hybrid model achieves an accuracy of 85.1 % on the CICIDS2017 dataset.
Kim et al. (Kim et al., 2014)	Hybrid IDS	Supervised	NSL-KDD	–	–	–	C4.5 decision tree	Multiple 1-class SVM	–	The hybrid model training and testing times in the second are 56.58 and 11.20, respectively.
Ozkan-Okay et al. (Ozkan-Okay et al., 2021)	Hybrid IDS	Supervised	KDD'99, UNSW-NB15	–	–	Proposed FSAP using pre-defined threshold	Rules extraction using basic features	Rules extraction using content features	–	The accuracy rate achieved by the proposed method is 99.17 %.
Kanna & Santhi (Kanna and Santhi, 2022)	Hybrid IDS	Supervised	NSL-KDD, UNSW-NB15, ISCX-IDS, CSE-CIC-IDS2018	–	Black Widow Optimization (BWO)	Artificial Bee Colony (ABC)	Black Widow Optimization-Convolutional neural network- Long short-term memory		BWO	98.66 % accuracy is observed in the case of the UNSW-NB15 dataset.
Khraisat et al. (Khraisat et al., 2019)	Hybrid IDS	Supervised	BoT-IoT	–	–	Information Gain	C5 decision tree	One-class SVM	–	The accuracy score achieved by the hybrid model is 99.97 %.
Proposed	Hybrid IDS	Supervised, Unsupervised, Reinforcement	UNSW-NB15, BoT-IoT, CICIoT2023	Fuzzy C-means	Modified GAMO by Integrating Attention Mechanism	Modified GWO by Integrating Correlation Coefficient	Majority voting classifier (DT, RF, ET, AB, LGBM, CB, XGB)	Double Deep Q-Network	OPTUNA	The accuracy scores are 98.31 % for SIDS and 94.74 % for AIDS models.

uses the generative adversarial minority oversampling technique with the fusion of the attention mechanism. Furthermore, the significant features of the dataset are selected using the nature-inspired modified grey wolf optimization approach by introducing the correlation coefficient and novel objective function. For known attack (or signature-based intrusion) detection, seven tree-based intelligent models are ensembled using a voting classifier and optimized with the OPTUNA technique. Reinforcement learning-based double deep Q-network is applied to detect unknown DoS attacks (or anomaly-based intrusion). Fig. 2 shows the overall architecture of the proposed framework, where all six tiers of the proposed model are presented.

3.1. Practical use case architecture

The statistics show an exponential growth of cyber-attacks in the IoT network every day after the COVID-19 pandemic. This is because everything is digitalized, such as online marketing, schooling, shopping, industry work in IT services, and even in the transportation, healthcare, and agriculture domains, and the proficiency of cyber attackers to perform their jobs wisely. This leads to dangerous and severe cyber-attacks that result in intensive loss at the organizational and individual levels. There is an urgent need to design a lightweight and hybrid NIDS that effectively secures IoT devices from diverse and contemporary cyber-attacks. IoT devices are highly susceptible to cyber-attacks due to their limited resources and the increasing number of security vulnerabilities they face. Traditional security measures such as firewalls and antivirus software are often insufficient to detect sophisticated cyber-attacks. Therefore, an accurate and robust lightweight NIDS (Network Intrusion Detection System) designed explicitly for IoT networks is critical.

In this paper, a lightweight hybrid network intrusion detection system (NIDS) is designed and installed in the IoT framework for deployment in a real-time scenario. Fig. 3 shows the NIDS deployment in the IoT framework. As depicted in the figure, there are two places where the attacker can take advantage and attack by sending malicious traffic: at the IoT devices (internal network traffic) and IoT gateway (external network traffic) levels. Here, the proposed NIDS is placed at the IoT device level and the IoT gateway level to secure them from harmful cyber-attacks. Algorithm 1 describes the overall step-by-step procedure of the proposed hybrid NIDS for IoT network security. D_{o_Train} and D_{o_Test} denote the original training and testing datasets, respectively.

3.2. Dataset description

This paper proposes a hybrid intrusion detection system designed to secure IoT devices. It utilizes three IoT-based datasets: UNSW-NB15 (Moustafa and Slay, 2015), BoT-IoT (Koroniotis et al., 2019), and CICIoT2023 (Neto et al., 2023). A brief explanation along with a tabular description of these datasets are provided in the following subsections:

3.2.1. UNSW-NB15 (Moustafa and Slay, 2015)

The dataset consists of 45 distinct features, including the target class. Among these features is the “attack_cat” attribute, which delineates different attack categories. Table 2 illustrates the training and testing samples distribution across the various attack and normal categories in the “attack_cat” column.

3.2.2. BoT-IoT (Koroniotis et al., 2019)

The BoT-IoT dataset was developed in a cyber-range lab at UNSW Canberra to simulate realistic botnet attack traffic in IoT networks. It consists of four sets: (i) Raw dataset: 70 GB of pcap files. (ii) Full dataset: 73 million instances in CSV format with 26 independent and three dependent features. (iii) 5 % subset: Contains 3.6 million instances and 43 independent features, organized into five CSV files. (iv) Final subset: Includes the same instances as the 5 % subset but focuses on the ten best features, comprising 19 features (16 independent and three dependent).

This paper uses the final subset, which is divided into “final 10-best training” and “final 10-best testing” CSV files (Peterson et al., 2021). Category-wise data distribution is shown in Table 3.

3.3.3. CICIoT2023 (Neto et al., 2023)

This paper discusses an IoT dataset with numerous records, analyzing a subset by selecting one subcategory for each of the seven attack categories: DDoS (SYN Flood), DoS (HTTP Flood), Recon (Ping Sweep), Web-Based (Backdoor Malware), Brute Force (Dictionary Brute Force), Spoofing (Arp Spoofing), and Mirai (UDPPlain). For the Normal category, the Benign Traffic3 subcategory is selected. The dataset has 40 features: one for the attack category and 39 for network traffic behavior. Data distribution for each category is in Table 4.

3.3. Dataset preprocessing

Effective data preprocessing is critical for optimizing the machine’s performance and deep learning models. The quality of data preprocessing significantly impacts the model’s ability to extract meaningful insights and patterns. This process involves removing redundant information, handling missing data, and transforming the data to be compatible with specific machine learning or deep learning algorithms. This stage also removes the irrelevant features from the data, such as the “id” and “label” features, which are present in the UNSW-NB15 dataset and are eliminated.

3.3.1. Label encoding

Label encoding converts each categorical feature in the data into appropriate numerical form. It maps each categorical value of a categorical column into the required numerical form. For example, if the protocol column of the data contains categories like TCP, UDP, and ICMP, label encoding maps these values into 0, 1, and 2, respectively.

3.3.2. Missing value imputation

There are many ways to impute the missing values in the data. This paper deals with the missing or null values in the data by replacing them with the mean value of the column. After the label encoding step, all the categorical columns (or features) within the data are converted into the numerical form. Thus, the mean of the particular column is calculated, and the null values in that column are replaced with the mean value of the column.

3.3.3. Clamp extreme values

Clamping is necessary for the extreme values to address skewness and manage outliers in the data. In this paper, if the maximum value of a column exceeds ten times the median value of the column, the column’s value is replaced with the 95th quantile value. If a column’s value is greater than the 95th quantile, the particular value is replaced with the column’s 95th quantile value; otherwise, the column values remain unchanged.

3.3.4. Feature scaling

In this paper, the original numeric data features are normalized using the standard scaler technique, which converts the mean and standard deviation of the data features to 0 and 1, respectively. Using this approach, the shape of the distribution of every feature vector is not changed; instead, it scales and shifts the distribution of the feature vector.

$$x_{scaled} = \frac{x_{original} - \text{mean}(\mu)}{\text{standard deviation}(\sigma)} \quad (1)$$

Where x_{scaled} represents the standardized feature, $x_{original}$ represents the initial feature, μ represents the mean of the feature, and σ denotes the standard deviation value.

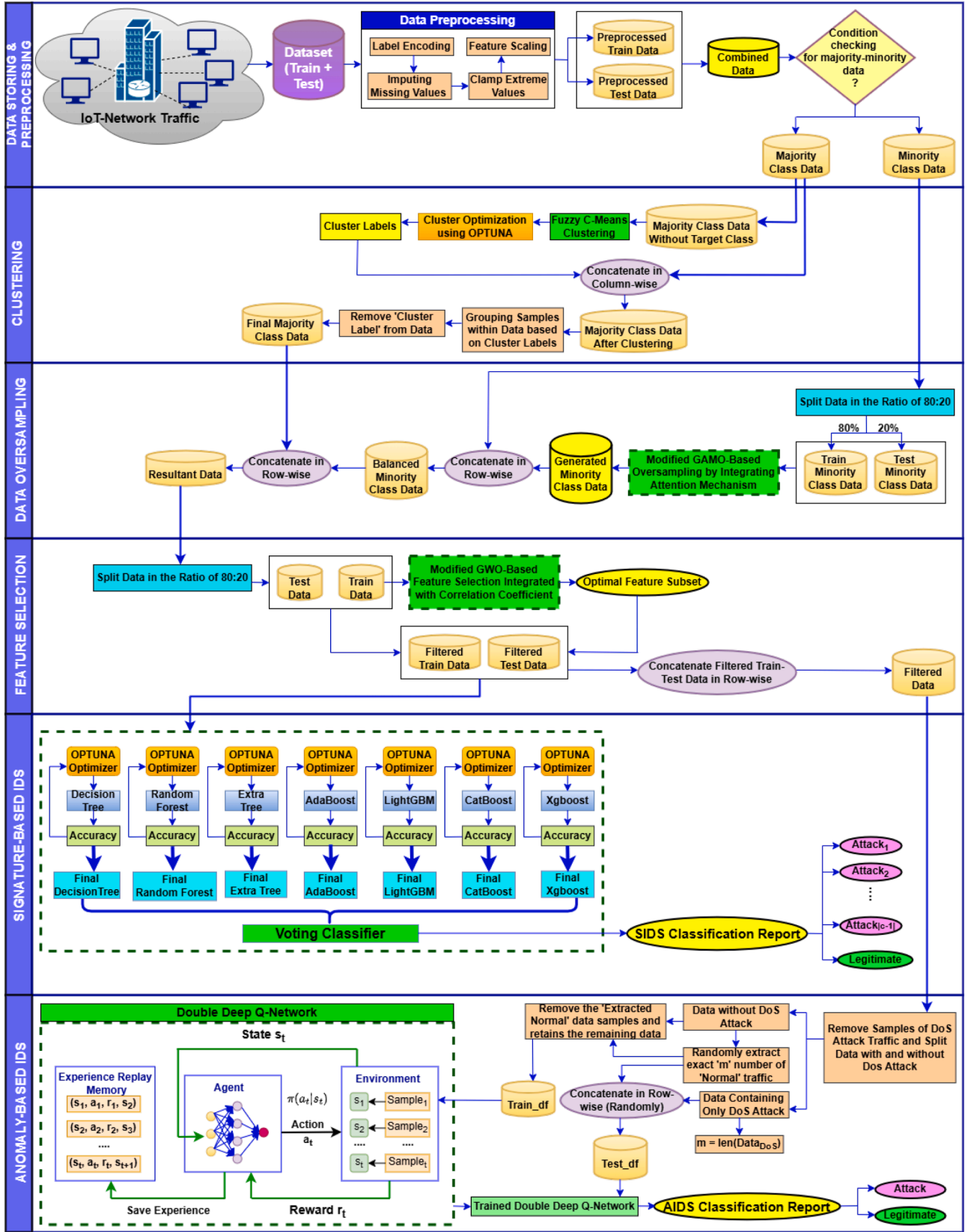


Fig. 2. Overall flow of proposed framework.

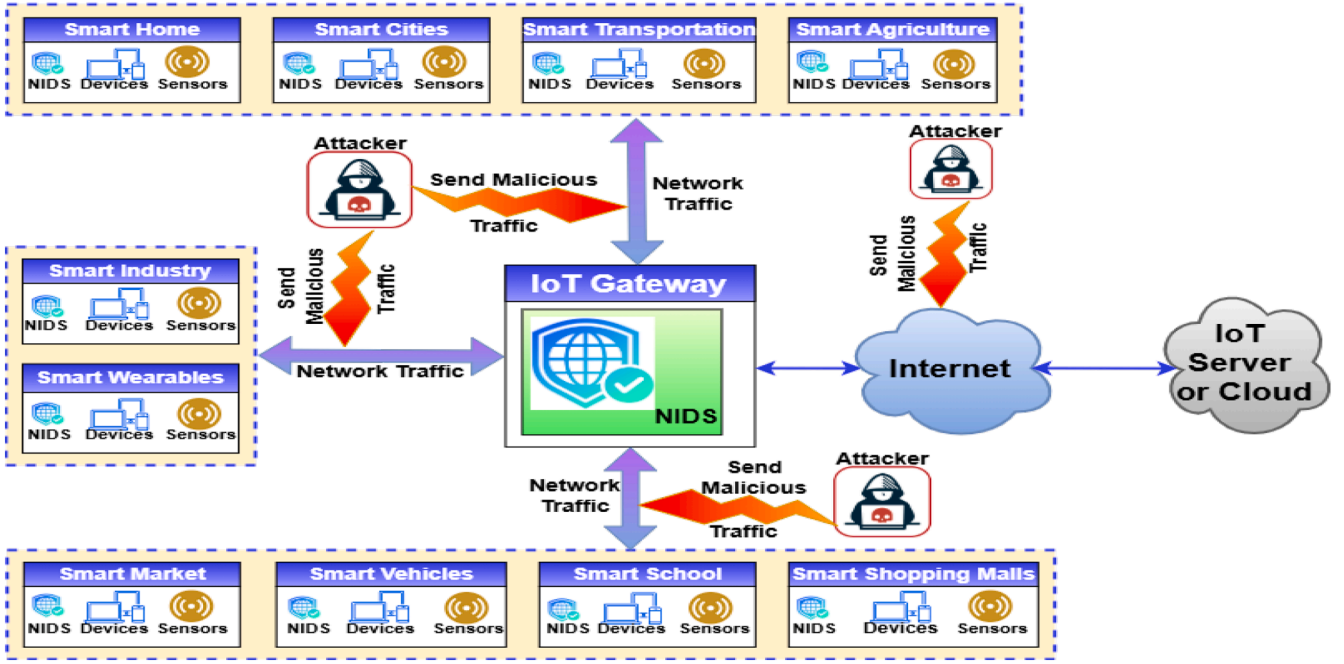


Fig. 3. NIDS in the IoT framework for deploying in real-time scenario.

Algorithm 1

Proposed hybrid NIDS for IoT network security.

```

Input: Original Dataset ( $D_{o\_Train}$ ,  $D_{o\_Test}$ )
Output: Hybrid NIDS Model
procedure Hybrid NIDS for IoT Network Security
2 begin
3   Combined_Data  $\leftarrow$  Concatenate ( $D_{o\_Train}$ ,  $D_{o\_Test}$ )
4   Preprocessed_Combined_Data  $\leftarrow$  Data Preprocessing (Combined_Data)
5    $D_{Major}$ ,  $D_{Minor}$   $\leftarrow$  Condition_Check (Preprocessed_Combined_Data)
6   Optimal_Hyperparameters  $\leftarrow$  OPTUNA-Based Hyperparameter Optimization
   (Fuzzy C-Means Clustering,  $D_{Major}$ )
7   Cluster_Labels  $\leftarrow$  Fuzzy C-Means Clustering ( $D_{Major}$ , Optimal
   Hyperparameters)
8   Final_Majority_Class_Data  $\leftarrow$  Grouping ( $D_{Major}$ , Cluster_Labels)
9   Generated_Minority_Class_Data  $\leftarrow$  Modified-GAMO-Based Data
   Oversampling ( $D_{Minor}$ )
10  Resultant_Data  $\leftarrow$  Concatenate (Final_Majority_Class_Data,
   Generated_Minority_Class_Data,  $D_{Minor}$ )
11   $D_{Train}$ ,  $D_{Test}$   $\leftarrow$  Train_Test_Split(Resultant_Data, Test_Size = 0.2)
12  fs  $\leftarrow$  Modified_GWO_Based_Feature_Selection ( $D_{Train}$ )
13   $D_{f\_Train}$ ,  $D_{f\_Test}$   $\leftarrow$  Filtering(fs,  $D_{Train}$ ,  $D_{Test}$ )
14  SIDS_Classification_Report  $\leftarrow$  Signature_Based_IDS ( $D_{f\_Train}$ ,  $D_{f\_Test}$ )
15  AIDS_Classification_Report  $\leftarrow$  Anomaly_Based_IDS ( $D_{f\_Train}$ ,  $D_{f\_Test}$ )
16 return SIDS_Classification_Report, AIDS_Classification_Report
15 end
16 end procedure

```

Table 2

Distribution of the UNSW-NB15 dataset across different categories.

Category	Training Instances	Testing Instances	Total Instances
Normal	56,000	37,000	93,000
Generic	40,000	18,871	58,871
Exploits	33,393	11,132	44,525
Fuzzers	18,184	6062	24,246
DoS	12,264	4089	16,353
Reconnaissance	10,491	3496	13,987
Analysis	2000	677	2677
Backdoor	1746	583	2329
Shellcode	1133	378	1511
Worms	130	44	174
Total	175,341	82,332	257,673

Table 3

Distribution of the BoT-IoT dataset across different categories.

Category	Training Instances	Testing Instances	Total Instances
DDoS	1541,315	385,309	1926,624
DoS	1320,148	330,112	1650,260
Reconnaissance	72,919	18,163	91,082
Normal	370	107	477
Information Theft	65	14	79
Total	2934,817	733,705	3668,522

Table 4

Distribution of the CICIoT2023 dataset across different categories.

Category	Training Instances (80 %)	Testing Instances (20 %)	Total Instances
DDoS	109,686	27,568	137,254
Normal	104,160	25,672	129,832
Spoofing	47,795	12,059	59,854
DoS	32,504	8182	40,686
Mirai	12,013	3077	15,090
Brute Force	10,501	2563	13,064
Web-Based	2569	649	3218
Recon	1780	482	2262
Total	321,008	80,252	401,260

3.3.5. Condition checking for majority-minority data

Conditions for distinguishing between majority and minority classes are determined using the following Eq. (2).

$$x = \begin{cases} \text{Major; if } (0.5 * |I_{major}|) \geq (|I_x|_{x \in \text{class}}) \text{ and } |I_{major}| = \max_{\forall x \in \text{class}} (|I_x|) \\ \text{Minor; Otherwise} \end{cases} \quad (2)$$

If the above mathematical inequality is satisfied by a specific class, "x", it simply means that class "x" belongs to the majority class category; otherwise, the class belongs to the minority class category. Where $|I_{major}|$ denotes the highest number of samples in the dataset, and $|I_x|$ represents the total number of samples of any specific class "x". Table 5 illustrates the majority and minority categories in all the datasets discussed in this

Table 5

Majority and Minority categories in the datasets.

Dataset	Majority Class	Minority Class
UNSW-NB15	Normal, Generic	Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor, Shellcode, Worms
BoT-IoT	DDoS, DoS	Reconnaissance, Normal, Information Theft
CIC-IoT2023	DDoS, Normal	Spoofing, DoS, Mirai, Brute Force, Web-Based, Recon

paper after applying the above Eq. (2). Only minority class data are provided to the modified GAMO-based oversampling module as input for generating the synthetic samples of only those minority categories and balancing the dataset.

3.4. Reducing class imbalance by oversampling: modified GAMO-Based oversampling by integrating attention mechanism

As shown in Tables 2, 3, and 4, it is evident that the IoT-based IDS dataset is highly imbalanced concerning different classes. Typically, a portion of the total data for each class is used to train the model, which can significantly influence model performance due to the class imbalance issue. The datasets exhibit a high imbalance; training and testing sets created by randomly selected samples also suffer from this imbalance. Consequently, this results in a biasness towards the majority classes in the training of any deep neural network model. In essence, the network fails to properly learn the minority classes due to weak representation caused by the inadequate number of samples.

A new version of the GAMO model (inspired by the GAMO model (Mullick et al., 2019)) is proposed in this paper to remove the minority class problem. Only minority class data is provided as input to this module, which is further split into training and testing datasets. The modified GAMO is designed to increase the number of samples from the minority classes through a generative adversarial approach. It generates the synthetic samples equivalent to the highest number of samples in the minority class category of the training datasets. By adding an attention layer to the generative adversarial minority oversampling (GAMO) model, this paper aims to enhance the model's performance and make

the model easily understandable. The modified GAMO generates additional samples from noise within the distribution of current class-specific minority samples. All the components of the modified GAMO framework are depicted in Fig. 4. The modified GAMO framework includes the following key elements:

- (i) Min-Max Scaling-based Feature Normalization unit processes original numerical categorical features and transforms them into a defined range.
- (ii) Extract the intermediate features from the noise using a conditional feature mapping unit.
- (iii) Generate new samples from the underrepresented (or minority) classes using a convex sample generator unit (SGU).
- (iv) Distinguishing genuine (real) from counterfeit (fake) samples using a conditional discriminator unit.
- (v) Performing the classification of the original (real) and the generated (fake) samples into categories using a classifier network unit.

3.4.1. Min-Max scaling-based feature normalization

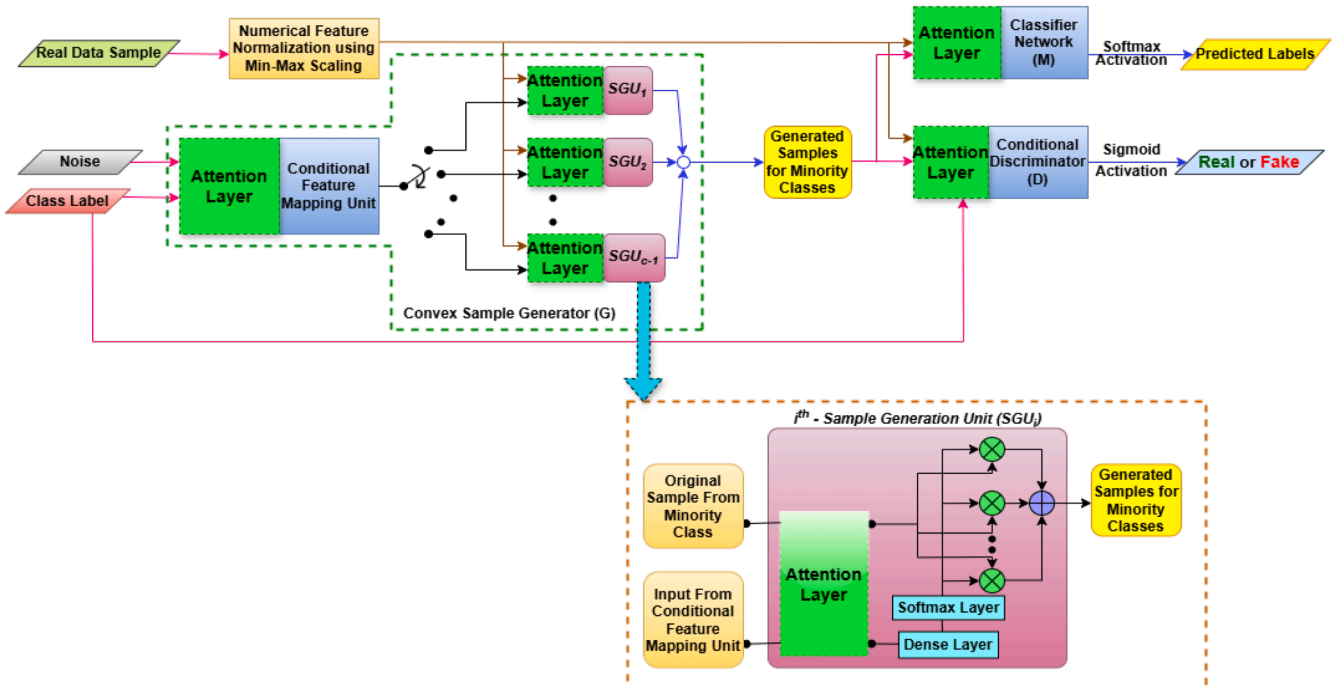
Since in the previous pre-processing module, only numerical features are scaled using the standard scaler technique. Thus, after the label encoding, all the categorical features are scaled. Hence, there is a high need to scale all the label-encoded features to keep them within a specific range and remove the outlier from the data.

$$\text{scaled}(x) = \frac{(x - \min(x))}{(\max(x) - \min(x))} \quad (3)$$

Where x is the original label-encoded features, $\min(x)$ represents the minimum value of the feature vector (x), $\max(x)$ denotes the maximum value of the feature vector (x), and $\text{scaled}(x)$ represents the normalized value of the feature vector (x) after min-max scaling which lies in the range of $[0,1]$. This is applied to both the training and testing data samples (Real Data Sample).

3.4.2. Conditional feature mapping unit

It generates intermediate features for minority classes using a

**Fig. 4.** Architecture of proposed generative model.

random vector noise (z) and class labels. Let “ r ” represent a random vector containing “ z ” non-negative real numbers, and “ l ” is the label vector with “ c ” values to denote the class labels, where “ c ” is the number of categories in the minority dataset. Only one value in “ l ” is set to one (corresponding to the class for which new samples are generated), while all other values are zero. This unit aims to map the random vector (r) to intermediate features conditioned on the label vector (l). In this process, the random vector (r) and label vector (l) are first concatenated to create a combined vector of size ($z + c$). For the UNSW-NB15 dataset, let $z = 32$ and $c = 8$; a combined vector of size 40 ($= 32+8$) is formed. A dense layer generates the features, with a total length of 32, followed by an Attention layer that generates features with dimensions 32. Subsequently, a dense layer of size 64 is added to the architecture. This process includes a ReLU activation function and batch normalization. The final step uses a dense layer of size 32 with ReLU activation function and batch normalization. The resulting output is the intermediate conditional feature vector. It is observed that the output of this conditional feature mapping unit connects to one of the convex SGUs based on the label (l). The detailed architecture and parameters of the conditional feature mapping unit are summarized in Table 6 for the UNSW-NB15 dataset.

3.4.3. Sample generation unit (SGU)

The data set contains the majority class with the highest number of samples in the training set, while the other classes are considered the minority. Fig. 4 illustrates the convex SGU, which oversamples the data points from minority classes. This means that ($c - 1$) SGU units are employed, with each minority class being assigned one unit. Where “ c ”

represents the total number of classes in the given data set. The i^{th} minority class’s SGU unit is denoted by $SGU_{i \in [1, c-1]}$. “ χ_i^g ” number of samples are generated by the SGU_i unit, such that

$$\chi_i^g = \chi_m - \chi_i, \quad \forall i \in [1, c-1] \quad (4)$$

Where χ_m and χ_i represent the number of training samples from the majority class and the number of training samples from the i^{th} minority class, respectively, any SGU unit takes two inputs, including the output of the conditional feature mapping unit and the real data samples. At a given time, only one SGU unit is enabled. The SGU_i is utilized when the sample belongs to the i^{th} minority class and when the i^{th} label is set. A dense layer, having the softmax activation function, transforms the intermediate feature into a length “ χ_i ”. Fig. 4 depicts the generator network (G) as the combination of the conditional feature mapping unit and the SGU. Table 7 describes the number of synthetic or fake samples generated by the generator neural network unit corresponding to a given minority class.

‘ χ ’ is a general term (for all classes), whereas ‘ χ_m ’ is a specific term (for the majority class). For a particular class ‘ i ’, ‘ χ_i ’ indicates the number of real train samples in that ‘ i^{th} ’ class. The number of synthetic samples generated by the proposed generative model is derived from the above eq. (4).

For the UNSW-NB15 dataset, the attack class ‘Exploit’ represents the majority class as it has the highest number of samples in the real training data. The number of samples of the ‘Exploit’ class is denoted by χ_m for this dataset. In eq. (4), ‘ i ’ can refer to any class within {Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor, Shellcode, and Worms}, and the corresponding number of synthetic samples generated for these classes is {0, 15,209, 21,129, 22,902, 31,393, 31,647, 32,260, and 33,263}, respectively, as derived from eq. (4). For instance, for the ‘DoS’ attack class, the number of samples generated is (χ^g) = 33,393 - 12,264 = 21,129.

Similarly, for the BoT-IoT dataset, the attack class ‘Reconnaissance’ represents the majority class as it has the maximum number of samples in the real training data. The number of samples of the ‘Reconnaissance’ class is denoted by χ_m for this dataset. In eq. (4), ‘ i ’ can refer to any class within {Reconnaissance, Normal, and Information Theft}, and the corresponding number of synthetic samples generated for these classes is {0, 72,549, and 72,854}, respectively, as derived from eq. (4). For instance, for the ‘Normal’ class, the number of samples generated is (χ^g) = 72,919 - 370 = 72,549.

Similarly, for the CIIoT2023 dataset, the attack class ‘Spoofing’ represents the majority class as it has the highest number of samples in the real training data. The number of samples of the ‘Spoofing’ class is denoted by χ_m for this dataset. In Eq. (4), ‘ i ’ can refer to any class within {Spoofing, DoS, Mirai, Brute Force, Web-Based, and Recon}, and the corresponding number of synthetic samples generated for these classes is {0, 15,291, 35,782, 37,294, 45,226, and 46,015}, respectively, as derived from Eq. (4). For instance, for the ‘Mirai’ attack class, the number of samples generated is (χ^g) = 47,795 - 12,013 = 35,782.

3.4.4. Conditional discriminator unit

To differentiate between the original sample and the generated one, created by the generator network (G), a discriminator is used to classify the original sample as “real” and the generated sample as “fake.” The details of the architectural parameters of the conditional discriminator neural network unit are given in Table 6. The network architecture consists of the following layers:

- An input layer to receive the original and generated sample of the minority class along with the label.
- Following the input layer, a dense layer with the number of neurons equal to the number of features in the data (here, for the UNSW-NB15 dataset, it equals 42) and a softmax activation function is added.

Table 6
Detailed parameters of the Modified GAMO on the UNSW-NB15 dataset.

Conditional feature mapping unit			
Layers	Size	Batch normalization	Activation function
Input Layer	(Noise+labels) = (32+8) = 40	–	–
Dense	32	–	–
Attention Layer	32	–	–
Dense	64	Yes	ReLU
Dense	32	Yes	ReLU
Sample Generation Unit (SGU)			
Input Layer	32	–	–
Dense	32	–	Softmax
Attention Layer	32	–	–
Dense	numMinor = 130; 1133; 1746; 2000; 10491; 12264; 18184	–	–
RepeatVector Layer	(42, 130); (42, 1133); (42, 1746); (42, 2000); (42, 10491); (42, 12264); (42, 18184)	–	–
Lambda	42	–	–
Conditional Discriminator			
Input Layer	(Input+labels) = (42+8) = 50	–	–
Dense	42	–	Softmax
Attention Layer	42	–	–
Dense	64	–	LeakyReLU
Dense	32	–	LeakyReLU
Dense	1	–	Sigmoid
Classifier Network (M)			
Input Layer	42	–	–
Dense	42	–	Softmax
Attention Layer	42	–	–
Dense	64	–	LeakyReLU
Dense	32	–	LeakyReLU
Dense	8 (No. of minority classes)	–	Softmax

Table 7

Description of the number of real training and corresponding generated samples.

UNSW-NB15			BoT-IoT			CICIoT2023		
Class	No. of real train samples (χ)	No. of generated (fake) samples (χ^g)	Class	No. of real train samples (χ)	No. of generated (fake) samples (χ^g)	Class	No. of real train samples (χ)	No. of generated (fake) samples (χ^g)
Exploits	33,393	0	Reconnaissance	72,919	0	Spoofing	47,795	0
Fuzzers	18,184	15,209	Normal	370	72,549	DoS	32,504	15,291
DoS	12,264	21,129	Information Theft	65	72,854	Mirai	12,013	35,782
Reconnaissance	10,491	22,902	-	-	-	Brute Force	10,501	37,294
Analysis	2000	31,393	-	-	-	Web-Based Recon	2569	45,226
Backdoor	1746	31,647	-	-	-	-	1780	46,015
Shellcode	1133	32,260	-	-	-	-	-	-
Worms	130	33,263	-	-	-	-	-	-

(iii) Following a dense layer, the attention layer is added to the network of size 42 (for the UNSW-NB15 dataset).

(iv) An attention layer is followed by three consecutive dense layers: The first layer has 64 neurons with a LeakyReLU activation function, and the second layer has 32 neurons with a LeakyReLU activation function, leading to a final dense layer with a sigmoid activation function.

(v) The final dense layer outputs the probability that the input sample is classified as real.

3.4.5. Classifier network

The modified GAMO network includes a classifier network (M) trained to classify samples and categorize newly generated synthetic samples validated by the discriminator. During training, the classifier network (M) uses original training samples of each class as well as the generated valid samples from the SGU of the minority classes. Testing involves evaluating the performance of the classifier network using testing data samples. The generator (G) and discriminator (D) networks are not employed during testing. Still, their use during training increases the learning capacity of the classifier network by including more samples for the minority classes generated within the boundary of those classes. The network architecture of the classifier network mirrors that of the discriminator network up to the second last layer, with the difference being the use of a dense layer with “c” outputs and a softmax classifier layer after the dense layer (second last layer) in the classifier network. The final dense layer produces class scores for each class, which are then used to compute the softmax (cross-entropy) loss. The details of the classifier network architecture and parameters are provided in Table 6 for the UNSW-NB15 dataset.

3.4.6. Objective function

The modified GAMO method involves three primary networks: generator (G), discriminator (D), and classifier (M). Training is carried out as a three-player minimax game; the objective function is formulated in the following Eq. (5):

$$\min_G \max_M \max_D O(G, M, D) = \sum_{i \in c} O_i \quad (5)$$

Where,

$$O_i = (O_{i1} + O_{i2} + O_{i3} + O_{i4} + O_{i5} + O_{i6}) \quad (6)$$

$$O_{i1} = P_i E_{x \sim p_i^d} [\log M_i(x)] \quad (7)$$

$$O_{i2} = \sum_{j \in c \setminus \{i\}} P_j E_{x \sim p_j^d} [\log(1 - M_i(x))] \quad (8)$$

$$O_{i3} = (P_c - P_i) E_{G(z|i) \sim p_i^g} [\log M_i(G(z|i))] \quad (9)$$

$$O_{i4} = \sum_{j \in c \setminus \{i\}} (P_c - P_j) E_{G(z|j) \sim p_j^g} [\log(1 - M_i(G(z|j)))] \quad (10)$$

$$O_{i5} = P_i E_{x \sim p_i^d} [\log D(x|i)] \quad (11)$$

$$O_{i6} = (P_c - P_i) E_{G(z|i) \sim p_i^g} [\log(1 - D(G(z|i)|i))] \quad (12)$$

The Eqs. (6) to (12) provide a clearer understanding of the objective function. The i^{th} class's prior probability is represented by P_i ; “x” and “G(z|i)” represent the original real and generated synthetic samples, respectively, and “c” represents the number of classes. p_i^d and p_i^g indicate the conditional probability of real and generated i^{th} class. The generator (G) network aims to trick the discriminator (O_{i6}). The discriminator (D) network is designed to identify the generated sample as bogus (O_{i6}) and the original sample as authentic (O_{i5}). The i^{th} class's class scores are maximized for the classifier neural network (M) concerning the original sample and the generated sample of the i^{th} class (O_{i1} and O_{i3}), while the class scores for the other classes (O_{i2} and O_{i4}) are minimized simultaneously.

3.5. Clustering: fuzzy C-Means clustering

Fuzzy C-Means (FCM) is a soft clustering technique that permits data points to be associated with several clusters, each with different membership levels. Unlike conventional clustering methods like K-Means, where each point is assigned to only one cluster, FCM assigns a membership value to each cluster for every point, reflecting how much it belongs to that cluster. This approach is especially helpful when the clusters are overlapping. This paper uses the fuzzy C-Means clustering algorithm to make the cluster of majority-class data samples. It is better than k-means clustering for the overlapped data. Here, only majority class data without target class is given as input to the algorithm. It is an unsupervised learning technique where the target class is not required. The formula for finding out the centroid of a cluster is given in the following Eq. (13):

$$V_{ij} = \frac{\sum_{k=1}^n \gamma_{ik}^m * x_k}{\sum_{k=1}^n \gamma_{ik}^m} \quad (13)$$

“V” is the centroid of a cluster, and it is computed for each cluster. “n” is the number of data points, “ γ ” is the fuzzy membership value, and “ x_k ” is the k^{th} data point in the dataset. “m” is the fuzziness parameter that determines the fuzziness level of the clustering, generally, $m > 1$. The mathematical formula for updating the fuzzy membership value is given in the following Eq. (14):

$$\gamma_{ki} = \left(\sum_{j=1}^n \left\{ \frac{d_{ki}^2}{d_{kj}^2} \right\}^{\left(\frac{1}{m-1} \right)} \right)^{-1} \quad (14)$$

Where “ d_{ki} ” is the distance between the data point and the cluster’s centroid, while “ i ” indicates the number of clusters.

3.5.1. Fuzzy C-Means clustering algorithm optimization: OPTUNA technique

The fuzzy C-means clustering algorithm’s hyperparameters are optimized using the OPTUNA-based hyperparameter tuning technique. Table 8 shows the optimal values of these hyperparameters for each dataset.

Table 9 shows the number of samples in the majority class before and after applying the fuzzy C-means clustering algorithm on each dataset.

Table 10 shows the details of the balanced dataset for each class after applying the proposed model’s oversampling and clustering techniques. The table shows that after applying the proposed data balancing technique, each category shows balanced behavior with respect to others in the dataset. This decreases bias towards the majority classes and provides an effective and fair performance of the hybrid IDS model.

3.6. Feature selection: modified GWO-Based feature selection integrated with correlation coefficient

Feature Selection is the crucial step for designing the hybrid IDS model. It is the process by which a subset of input features can be selected from the IoT network data traffic to build an intelligent IDS model. The main intention of feature selection is to obtain the optimal feature subset from the data traffic, which accurately represents the given problem and enables the construction of a robust hybrid IDS model with high performance, strength, and detection accuracy. It also reduces the model’s complexity, which avoids common problems in detecting suspicious known and unknown cyber-attacks, such as high dimensionality of the data, prediction time of suspicious behavior, computational ineffectiveness, and model understanding. Avoiding the feature selection process may lead to a vulnerable model with low performance, strength, limited model understandability, and high false alarm rates, leading to higher utilization of available resources in the IoT network framework.

This paper uses a modified form of the Grey Wolf Optimization (GWO) algorithm for the feature selection task. Mirjalili et al. (Mirjalili et al., 2014) have developed the GWO algorithm in 2014. The proposed flow chart of the modified grey wolf optimization-based feature selection is provided in the following Fig. 5. The step-by-step procedure of the proposed grey wolf optimization-based feature selection algorithm is described by Algorithm 2. Table 11 depicts the parameter settings utilized by the proposed feature selection algorithm. As observed in Table 11, very few parameters are present in the modified GWO-based feature selection algorithm. Fewer parameters of the GWO model reduce the model’s time complexity. Only the training dataset is provided as input to this module.

3.6.1. Initialization of population

Initialization of the population is the first step in any of the meta-heuristic algorithms. In this case, the population of the grey wolf optimization algorithm is initiated for the feature selection task; therefore, each wolf in the population represents the candidate solution. Here, the population is randomly initialized with binary values 0 and 1. The structure of a wolf is shown in Fig. 6 with a green color, where “1” in a wolf position represents the presence of the corresponding feature, and

“0” represents the absence of the respective feature in the candidate solution (or wolf position). $f_1, f_2, f_3, \dots, f_n$ represent the features in the dataset, and $w_1, w_2, w_3, \dots, w_n$ represent the wolf numbers. For each wolf in the population, their respective correlation is computed by applying the Eq. (15). For the i^{th} wolf, the correlation value is indicated by Corr_wi in Fig. 6. After computing the correlation value for each wolf in the population, the condition is scrutinized whether the correlation value of the wolf is lesser than 0.6. If the following condition is satisfied, add the corresponding wolf to the improved population; otherwise, remove that wolf from the population. Function 1 describes the step-by-step procedure of the modified initialization of the population in the grey wolf optimization-based feature selection.

Where $F_1, F_2, F_3, \dots, F_n$ represent features, and “ n ” is the total number of features. $\text{COV}(F_1, F_i)$ is the covariance between features F_1 and F_i . “ σ_{F_i} ” is the standard deviation of feature “ F_i ”. Function 2 describes the process of computing the correlation of each wolf. The following Eqs. (15) to (19) describe the formula for measuring the correlation of each wolf.

$$\text{correlation} = \frac{\sum_{i=1}^n \text{cov}(F_1, F_i)}{n \sum_{i=1}^n \sigma_{F_i}} \quad (15)$$

$$|F_1| = |F_2| = |F_3| = \dots = |F_n| = |F| \quad (16)$$

$$\text{cov}(F_m, F_n) = \frac{1}{|F|} * \sum_{i=1}^{|F|} (F_{mi} - E(F_m)) * (F_{ni} - E(F_n)) \quad (17)$$

$$\sigma_F = \sqrt{\frac{\sum_{i=1}^{|F|} (f_i - E(F))^2}{|F|}} \quad (18)$$

$$E(F) = \frac{f_1 + f_2 + f_3 + \dots + f_{|F|}}{|F|} = \frac{\sum_{i=1}^{|F|} f_i}{|F|} \quad (19)$$

3.6.2. Objective function (or fitness function) evaluation

The effectiveness of each candidate solution (wolf) is measured using the fitness or objective function. The objective function is utilized to assess the fitness of each wolf based on its location. This paper uses a novel fitness or objective function, a combination of the min-max term. The following Eq. (20) shows the formula for computing the fitness score of each wolf. To measure the accuracy, error rate, and FAR, this paper uses the XGBOOST algorithm and performs the multiclass classification. The following Eqs. (21), (22), and (23) give the formula for measuring accuracy, error_rate, and FAR, respectively. The Eq. (20) shows that the objective function combines the min-max term. Some parts of the equation maximize the objective function, while others minimize it. The primary intent of the fitness function is to maximize the term in the square bracket. Fig. 7 demonstrates an example of how the optimal feature subset is selected by employing the proposed method only for seven iterations.

$$\text{fitness_score} = \max \left[\text{Accuracy} - \min \left\{ \alpha * (\text{Error_Rate} + \text{FAR}) + (1 - \alpha) * \left(\frac{\text{Feature subset}}{\text{Total features}} \right) \right\} \right] \quad (20)$$

$$\text{Accuracy} = \frac{\text{Correctly Predicted Instances}}{\text{Overall Number of Instances}} \quad (21)$$

$$\text{Error_Rate} = (1 - \text{Accuracy}) \quad (22)$$

$$\text{FAR} = \frac{\text{False Positive (FP)}}{\text{False Positive (FP)} + \text{True Negative (TN)}} \quad (23)$$

Where “ α ” is a weighting parameter, and by the trial and error method, its value equals 0.5. The feature subset represents the candidate

Table 8

Optimal Hyperparameter values of Fuzzy C-Means Clustering algorithm.

Hyperparameter	UNSW-NB15	BoT-IoT	CICIoT2023
Number of Clusters (c)	166	183	210
Fuzziness Coefficient (m)	1.013	1.098	1.199
Convergence Threshold (error)	0.0005	0.001	0.005
Max. No. of iterations (maxiter)	1025	1156	1200

Table 9

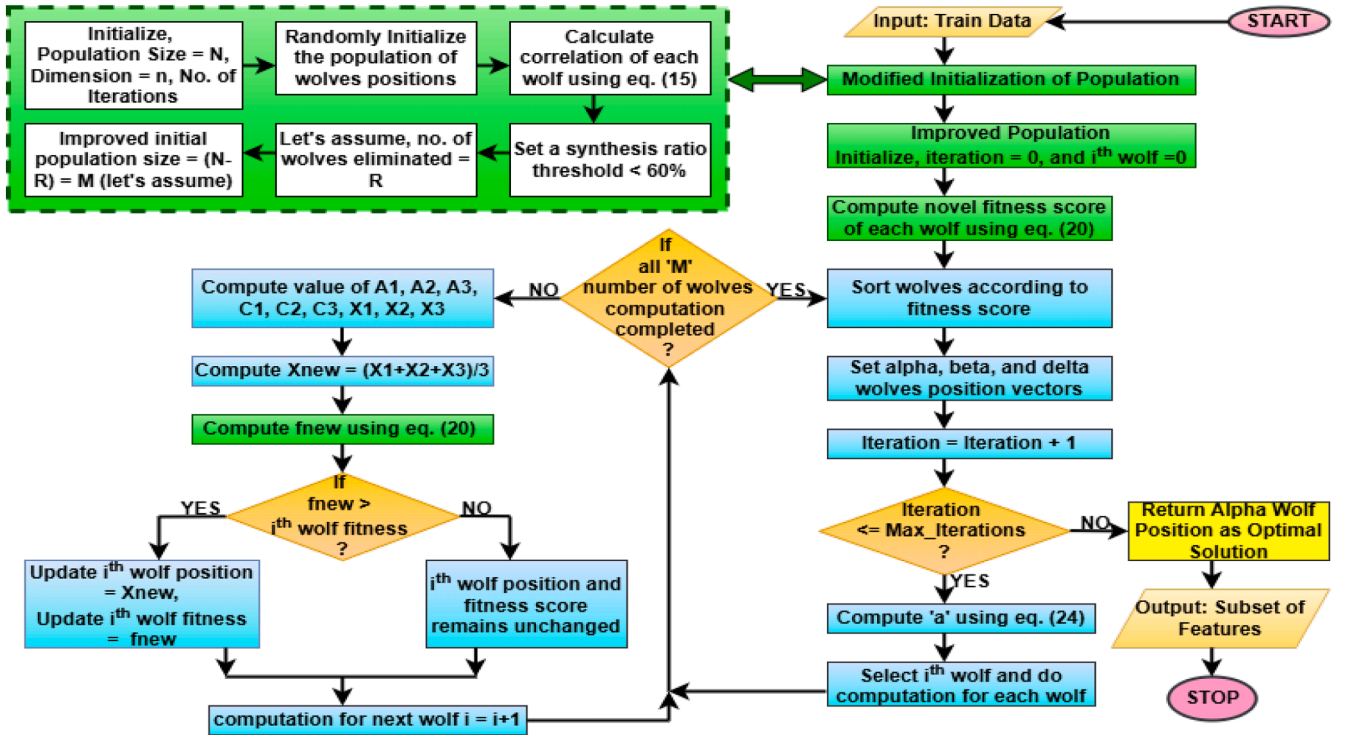
Number of samples in majority class before and after applying clustering.

UNSW-NB15			BoT-IoT			CICIoT2023		
Majority Class	Before (Real)	After (Reduced)	Majority Class	Before (Real)	After (Reduced)	Majority Class	Before (Real)	After (Reduced)
Normal	93,000	80,912	DDoS	1926,624	92,000	DDoS	137,254	30,439
Generic	58,871	31,204	DoS	1650,260	92,000	Normal	129,832	90,002

Table 10

Details of balanced dataset.

UNSW-NB15		BoT-IoT		CICIoT2023	
Category	Total Instances	Category	Total Instances	Category	Total Instances
Normal	80,912	DDoS	92,000	DDoS	30,439
Generic	31,204	DoS	92,000	Normal	90,002
Exploits	44,525	Reconnaissance	91,082	Spoofing	59,854
Fuzzers	39,455	Normal	73,026	DoS	55,977
DoS	37,482	Information Theft	72,933	Mirai	50,872
Reconnaissance	36,889	-	-	Brute Force	50,358
Analysis	34,070	-	-	Web-Based	48,444
Backdoor	33,976	-	-	Recon	48,277
Shellcode	33,771	-	-	-	-
Worms	33,437	-	-	-	-
Total	405,721	-	421,041	-	434,223

**Fig. 5.** Flow chart of modified GWO-based feature selection integrated with correlation coefficient.

solution of the corresponding wolf, and the total feature represents the total number of features in the training dataset. [Function 3](#) describes the step-by-step process of computing the fitness score of each wolf (candidate solution). Each wolf is arranged in descending order based on their fitness score. The wolf with the highest fitness score (the first wolf) is expressed as an alpha wolf; following this, the second wolf is considered the beta wolf. Subsequently, the third wolf is regarded as the delta, and the fourth wolf is considered the omega wolf.

3.6.3. Updating positions

The positions of the wolves are updated in each iteration to make

changes in the population. The updated wolf's position is better than the previous one. This process is continued in each iteration until the optimal position is achieved. The value of "a" is computed using the formula given in [Eq. \(24\)](#) and updated for each iteration. The value of "a" ranges between 0 and 2. Initially, it is near two and linearly decreases its values to 0. It is a parameter that controls the balance between exploitation and exploration.

$$a = 2 * \left(1 - \frac{\text{current_iteration}}{\text{maximum no. of iterations}} \right) \quad (24)$$

At the time "t", the positions of alpha, beta, and delta wolves are

Algorithm 2

Modified grey wolf optimization.

Input: Train_Data, Population size $\leftarrow$ 50, No. of Dimensions $\leftarrow$ No. of features, No. of Iterations $\leftarrow$ 10
Output: Optimal subset of features

procedure

- begin**
- 1 Initialize, iteration $\leftarrow$ 0, fitness $\leftarrow$ [], new_wolf $\leftarrow$ [], new_fitness $\leftarrow$ []
- 2 Improved_Population $\leftarrow$ Modified Initialization of Population (Population size, No. of Dimensions, No. of Iterations)
- 3 **for** i in range (len (Improved_Population)):
- 4 fitness_wolf[i] $\leftarrow$ Compute_fitness(wolf[i], Train_Data)
- 5 fitness.append(fitness_wolf[i])
- 6 **end for**
- 7 sorted_wolf_positions $\leftarrow$ sort wolf positions according to fitness in descending order
- 8 alpha_position $\leftarrow$ sorted_wolf_positions[0]
- 9 beta_position $\leftarrow$ sorted_wolf_positions[1]
- 10 delta_position $\leftarrow$ sorted_wolf_positions[2]
- 11 **for** i in range (No. of Iterations):
- 12 $a \leftarrow 2 * \left(1 - \frac{i}{\text{No. of Iterations}}\right)$
- 13 **for** j in range (len(Improved_Population)):
- 14 $A_1 = 2 * a * \text{rand} - a, A_2 = 2 * a * \text{rand} - a, A_3 = 2 * a * \text{rand} - a$
- 15 $C_1 = 2 * \text{rand}, C_2 = 2 * \text{rand}, C_3 = 2 * \text{rand}$
- 16 $X_1 = \text{alpha_position} - A_1 * \text{abs}(C_1 * \text{alpha_position} - \text{wolf}[j])$
- 17 $X_2 = \text{beta_position} - A_2 * \text{abs}(C_2 * \text{beta_position} - \text{wolf}[j])$
- 18 $X_3 = \text{delta_position} - A_3 * \text{abs}(C_3 * \text{delta_position} - \text{wolf}[j])$
- 19 $X_{\text{new}} = \frac{X_1 + X_2 + X_3}{3}$
- 20 $X_{\text{normalized}} \leftarrow \text{Normalized}(X_{\text{new}})$
- 21 $f_{\text{new}} \leftarrow \text{Compute_fitness}(X_{\text{normalized}}, \text{Train_Data})$
- 22 **if** ($f_{\text{new}} > \text{fitness_wolf}[j]$):
- 23 $\text{wolf}[j] \leftarrow X_{\text{normalized}}$
- 24 $\text{fitness_wolf}[j] \leftarrow f_{\text{new}}$
- 25 **end if**
- 26 new_wolf.append(wolf[j])
- 27 new_fitness.append(fitness_wolf[j])
- 28 **end for**
- 29 Improved_Population $\leftarrow$ new_wolf
- 30 sorted_wolf_positions $\leftarrow$ sort new_wolf positions according to new_fitness in descending order
- 31 alpha_position $\leftarrow$ sorted_wolf_positions[0]
- 32 beta_position $\leftarrow$ sorted_wolf_positions[1]
- 33 delta_position $\leftarrow$ sorted_wolf_positions[2]
- 34 **end for**
- 35 Optimal subset of features $\leftarrow$ alpha_position
- 36 **return** Optimal subset of features
- 37 **end**
- 38 **end procedure**

Table 11

Parameter settings of modified grey wolf optimization algorithm for feature selection.

Parameter	Value
Population size	20 (UNSW-NB15), 50 (BoT-IoT), 30 (CIC-IoT2023)
No. of iterations	86 (UNSW-NB15), 95 (BoT-IoT), 112 (CIC-IoT2023)
Dimension (No. of Features) excluding target class	42 (UNSW-NB15), 18 (BoT-IoT), 39 (CIC-IoT2023)
Seed	40
rand	[0,1]
α	0.5

achieved, which are denoted by $X_\alpha(t)$, $X_\beta(t)$, and $X_\delta(t)$ respectively. For computing the new position of a wolf at the time $(t + 1)$, the positions of $X_1(t)$, $X_2(t)$, and $X_3(t)$ need to be computed first at the time “t”. Eqs. (25)–(36) are the values of the parameters that need to be computed before updating the new position of a wolf where $D_\alpha(t)$, $D_\beta(t)$, and $D_\delta(t)$ indicate the distance between the current wolf position and alpha, beta, and delta wolves, respectively at the time “t”.

$$A_1 = 2 * a * \text{rand} - a \quad (25)$$

$$C_1 = 2 * \text{rand} \quad (26)$$

$$D_\alpha(t) = |C_1 * X_\alpha(t) - X(t)| \quad (27)$$

$$X_1(t) = X_\alpha(t) - A_1 * D_\alpha(t) \quad (28)$$

$$A_2 = 2 * a * \text{rand} - a \quad (29)$$

$$C_2 = 2 * \text{rand} \quad (30)$$

$$D_\beta(t) = |C_2 * X_\beta(t) - X(t)| \quad (31)$$

$$X_2(t) = X_\beta(t) - A_2 * D_\beta(t) \quad (32)$$

$$A_3 = 2 * a * \text{rand} - a \quad (33)$$

$$C_3 = 2 * \text{rand} \quad (34)$$

$$D_\delta(t) = |C_3 * X_\delta(t) - X(t)| \quad (35)$$

$$X_3(t) = X_\delta(t) - A_3 * D_\delta(t) \quad (36)$$

Where A_1 , A_2 , A_3 , C_1 , C_2 , and C_3 remain constants for an iteration, and their values are updated due to the updation of the value of “a” in each iteration, and $X(t)$ is the position of any wolf at time “t”. The new position of a wolf is computed at the time “t + 1”, is just an average of the positions of $X_1(t)$, $X_2(t)$, and $X_3(t)$, as shown in the following Eq. (37):

$$X_{\text{new}}(t + 1) = \frac{X_1(t) + X_2(t) + X_3(t)}{3} \quad (37)$$

3.6.4. Handling boundary values

To make the updated positions of the wolves within a range of binary numbers means their values are either 0 or 1; this paper normalizes the values to make them binary. For each wolf in the population, if the value at any index is greater than 0.5, it is transformed into 1, and if the value is < 0.5 , it is converted into 0. It is performed to make the presence and absence of the features index within a wolf.

3.6.5. Termination criterion

When the best fitness score of the wolves stops increasing their values, it is the condition for terminating the algorithm. This means that after a certain number of iterations, the value of the fitness score is not enhanced; this is the optimal value of the fitness score of a particular wolf. The wolf’s corresponding optimal position is considered the best feature subset. The trial and error method determines the different number of iterations for each dataset. The number of iterations for the UNSW-NB15, BoT-IoT, and CICIoT2023 datasets are 86, 95, and 112, respectively. Table 12 describes the dataset details after the feature selection module of the proposed model, where each dataset is split into training and testing sets in the ratio of 80:20.

3.7. Hybrid IDS framework

The Hybrid IDS framework combines the signature-based IDS (SIDS) and anomaly-based IDS (AIDS) frameworks. Using intelligent models, this hybrid framework can detect known and unknown attack categories efficiently. The signature-based IDS uses the tree-based approach to detect known attack behavior in IoT-based processed network traffic. However, the anomaly-based IDS uses the advanced Q-learning technique to detect the unknown attack from the IoT-based processed network traffic. This module uses the filtered dataset after the feature selection module of the proposed framework. The following sections, 3.7.1 and 3.7.2, discuss signature-based and anomaly-based IDS frameworks, respectively.

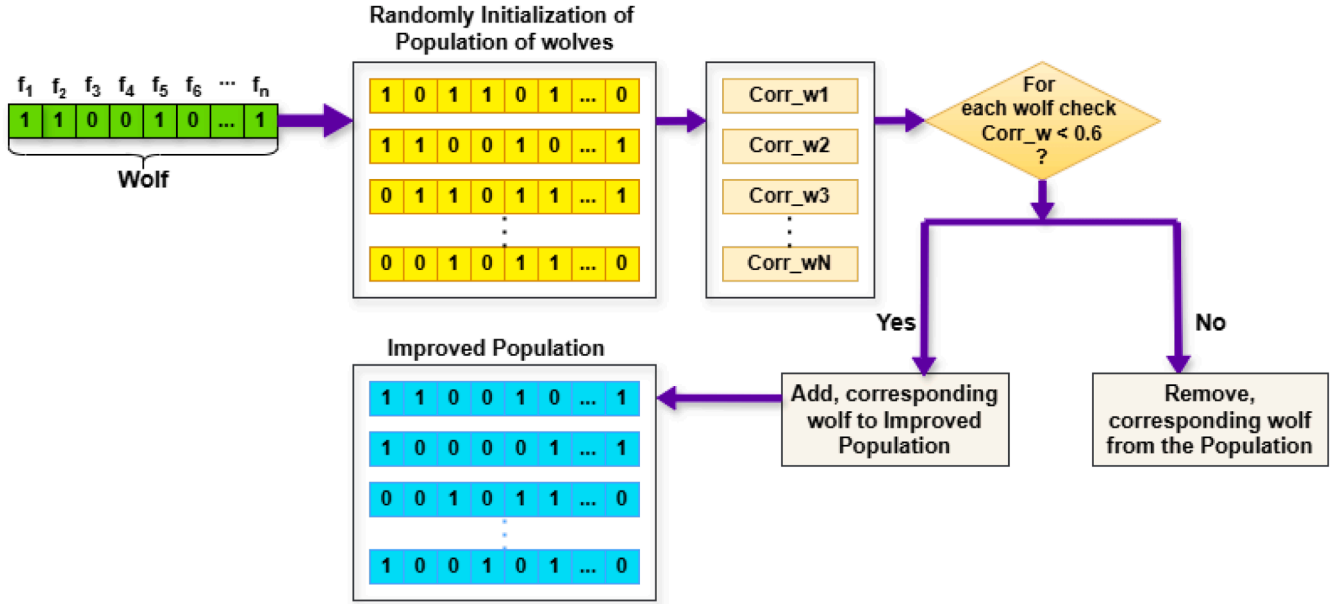


Fig. 6. Structure of wolves, assembly of wolves in a population, procedure of constructing improved population.

Function 1

Modified initialization of population (population size, No. of dimensions, No. of iterations).

```

Input: Population size  $\leftarrow N$ , No. of Dimensions  $\leftarrow n$ , No. of Iterations  $\leftarrow 50$ 
Output: Improved_Population
procedure
1  begin
2    Improved_Population  $\leftarrow []$ , Wolf_Correlation  $\leftarrow []$ 
3    Wolves_Population  $\leftarrow \text{random}(\text{Population size, No. of Dimensions})$ 
4    for  $i$  in range(len(wolves_population)):
5      Corr_wolf $i$   $\leftarrow \text{Calculate Correlation}(\text{wolf}_i, D_{\text{Train}})$ 
6      Wolf_correlation.append(Corr_wolf $i$ )
7    end for
8    for  $i$  in range(len(Wolf_correlation)):
9      if (Corr_wolf $i$  < 0.6):
10       Improved_Population.append(wolf[i])
11     end if
12   else:
13     Print("Feature subset within a wolf[i] is highly correlated!!")
14   end else
15 end for
16 return Improved_Population
17 end
18 end procedure

```

Function 2

Calculate correlation(wolf _{i} , D_{Train}).

```

Input: wolf $i$ , DTrain
Output: corr_wolf $i$ 
procedure
1  begin
2    Cov = [], SD = []
3    for  $i \leftarrow 2$  to len(wolf $i$ )
4      Cov(F1, F $i$ )  $\leftarrow \text{Compute Covariance between } F_1, \text{ and } F_i$ 
5      SD(F $i$ )  $\leftarrow \text{Compute Standard Deviation}(F_i)$ 
6      Cov.append(Cov(F1, F $i$ ))
7      SD.append(SD(F $i$ ))
8    end for
9     $\text{Corr\_wolf}_i \leftarrow \frac{\text{sum}(Cov)}{n * \text{sum}(SD)}$ 
10   return Corr_wolf $i$ 
11 end
12 end procedure

```

3.7.1. Signature-based intrusion detection system framework

Since no deep learning model performs better than any of the tree-based classifiers in the IDS paradigm (Zoppi et al., 2024), this paper uses the ensemble of seven different tree-based classifiers for known attack detection. These seven different tree-based classifiers used in this paper are as follows: Decision Tree (DT), Random Forest (RF), Extra Trees (ET), Adaboost (AB), LightGBM (LGBM), Catboost (CB), and Xgboost (XGB). The hyperparameters of these classifiers are optimized using the OPTUNA-based technique, as discussed in the following sub-Section 3.7.1 (i). Algorithm 3 describes the step-by-step procedure for designing the SIDS model for IoT networks. The ensembling of these classifiers is carried out using the majority voting classifier. For this module, the filtered dataset is split into training and testing datasets in the ratio of 80:20. Here, the multi-class classification report is generated by the proposed SIDS model, which classifies the normal and attack categories and the different attack categories.

(i) Hyperparameter Optimization of Classification Algorithm of SIDS using OPTUNA Technique

The hyperparameters of these seven machine-learning-based classifiers are optimized using the OPTUNA-based hyperparameter tuning techniques (Akiba et al., 2019). This technique is iterated in different trials for different datasets. This paper checks the accuracy value of each detection model after each trial, and it is observed that after a certain number of trials, the accuracy value stops enhancing. So, the obtained hyperparameter values are the optimal hyperparameter sets for those detection models. The number of trials for UNSW-NB15, BoT-IoT, and CICIOT2023 datasets is 104, 128, and 164, respectively. The filtered train and filtered test data traffic of each dataset are given as input to this module, and it provides the best hyperparameter sets corresponding to each detection model. Table 13 shows the optimal hyperparameter values of each detection model before applying the majority voting classifier.

3.7.2. Anomaly-based intrusion detection system (AIDS) framework

An anomaly-based intrusion detection system framework is used for unknown DoS attack detection, where the filtered dataset is given as input to this module. This module applies the reinforcement learning-based double deep Q network model. For unknown attack detection, the primary intent of the paper is to train the model with a different set

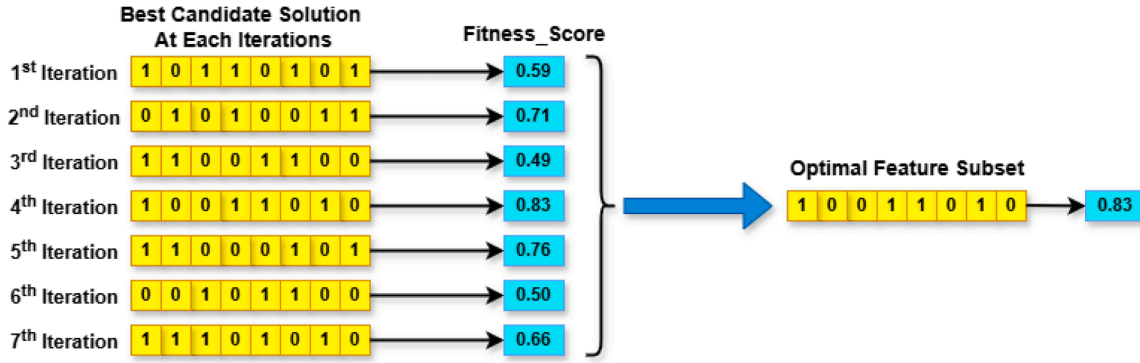


Fig. 7. Optimal feature subset selection utilizing local search for seven iterations.

Function 3Compute_fitness($X_{\text{normalized}}$, train_data).

```

Input:  $X_{\text{normalized}}$ , Train_Data,  $\alpha$ 
Output: fitness_score
procedure
1  begin
2  Initialize, count  $\leftarrow$  0
3  Accuracy  $\leftarrow$  Accuracy(XGBOOST( $X_{\text{normalized}}$ , Train_Data))
4  Error_Rate  $\leftarrow$  Error_Rate(XGBOOST( $X_{\text{normalized}}$ , Train_Data))
5  FAR  $\leftarrow$  FAR(XGBOOST( $X_{\text{normalized}}$ , Train_Data))
6  total_features  $\leftarrow$  Train_Data.shape[1] - 1
7  for i in range(len( $X_{\text{normalized}}$ )):
8    if ( $X_{\text{normalized}}[i] == 1$ ):
9      count  $\leftarrow$  count + 1
10   end if
11 end for
12 feature_subset  $\leftarrow$  count
13 subset_ratio  $\leftarrow$  (feature_subset) / total_features
14 fitness_score  $\leftarrow$  Accuracy -  $\{\alpha * (\text{Error\_Rate} + \text{FAR}) + (1-\alpha) * \text{subset\_ratio}\}$ 
15 return fitness_score
16 end
17 end procedure

```

Table 12

Dataset description after feature selection.

ataset	UNSW-NB15	BoT-IoT	CICIoT2023
Number of training samples	324,577	336,833	347,378
Number of testing samples	81,144	84,208	86,845
Number of reduced features	25	10	21

of IoT-based network traffic and test it using a completely different set of IoT-based network traffic.

For each dataset used in this paper, one attack category is common in all three: the DoS attack. Therefore, this paper detects the IoT-based unknown DoS attack traffic. Now, there are different attack categories in each dataset. Since this paper detects the unknown DoS attack traffic within the dataset, each DoS attack traffic is extracted from the filtered dataset. Now, store this DoS attack traffic of each dataset in another data frame for the testing purpose of the trained double deep q network model.

(i) Double Deep Q Network (DDQN)

The DDQN model enhances the deep q-network (DQN) algorithm, combining reinforcement learning and deep learning techniques. This integration has led to significant progress in addressing complex problems. The architecture of the DDQN model is illustrated in Fig. 8. The model is based on the Q-learning algorithm, which learns the Q-function to estimate expected future rewards for specific actions in a given state. The model progressively learns the optimal decision-making strategy by

Algorithm 3

Signature_based_IDS.

```

Input: Filtered Dataset ( $D_{f\_Train}$ ,  $D_{f\_Test}$ )
Output: SIDS_Classification_Report
procedure Signature-Based Attack Detection for securing IoT Networks
1  begin
2   $x_{\text{train}}$ ,  $y_{\text{train}} \leftarrow D_{f\_Train}.\text{drop}(\text{'attack\_cat'}), D_{f\_Train}.\text{'attack\_cat'}$ 
3   $x_{\text{test}}$ ,  $y_{\text{test}} \leftarrow D_{f\_Test}.\text{drop}(\text{'attack\_cat'}), D_{f\_Test}.\text{'attack\_cat'}$ 
4  Model  $\leftarrow$  {DT, RF, ET, AB, LGBM, CB, XGB}
5  for each M in Model:
6    OH_M  $\leftarrow$  OPTUNA-Based Hyperparameter Tuning (M,  $D_{f\_Train}$ ,  $D_{f\_Test}$ )
7  end for
8  DT  $\leftarrow$  DecisionTreeClassifier(OH_DT)
9  RF  $\leftarrow$  RandomForestClassifier(OH_RF)
10 ET  $\leftarrow$  ExtraTreesClassifier(OH_ET)
11 AB  $\leftarrow$  AdaBoostClassifier(OH_AB)
12 LGBM  $\leftarrow$  LGBMClassifier(OH_LGBM)
13 CB  $\leftarrow$  CatBoostClassifier(OH_CB)
14 XGB  $\leftarrow$  XGBClassifier(OH_XGB)
15 Majority_Voting  $\leftarrow$  VotingClassifier(DT, RF, ET, AB, LGBM, CB, XGB, voting = 'hard')
16 Majority_Voting.fit( $x_{\text{train}}$ ,  $y_{\text{train}}$ )
17 Predictions  $\leftarrow$  Majority_Voting.predict( $x_{\text{test}}$ )
18 SIDS_Classification_Report  $\leftarrow$  classification_report( $y_{\text{test}}$ , Predictions)
19 return SIDS_Classification_Report
20 end
21 end procedure

```

updating the Q-function based on observed rewards. The DDQN model incorporates two neural networks: the main network, which predicts Q-values for each action in a given state, and the target network, which provides a more consistent and accurate Q-value estimate. The target network's weights are periodically updated from the main network to reduce overestimation bias. Here, the DDQN model is applied to the binary classification task. Initially, all "DoS" attack data samples are removed from the training dataset and appended to the testing dataset. The exact number of normal samples (such as DoS attack data samples) are removed from the training dataset and appended to the testing dataset. The classification problem is formulated: the training data includes normal network behavior and known attack categories (excluding DoS attacks). In contrast, the testing data contains normal and previously unknown DoS attack traffic. The double DQN is employed to correctly classify normal traffic and identify DoS attack traffic as anomalous, even though it has not been trained on DoS attacks. Using double DQN for unknown DoS attack detection poses a challenging anomaly detection problem due to the absence of DoS attack instances in the training data. This resembles zero-day (or zero-shot) anomaly detection, where the agent must generalize to detect a previously unknown attack category (here, DoS attack) using knowledge gained from other types of attacks and normal data.

The key challenges include training the agent to identify anomalies in the testing phase and generalizing its understanding of anomalous

Table 13
Optimal Hyperparameter values of different detection models for the voting classifier.

Detection Model	Best Hyperparameters		
	UNSW-NB15	BoT-IoT	CICIoT2023
DT	splitter: random criterion: gini max_depth: 12 min_samples_split: 10 min_samples_leaf: 14 min_weight_fraction_leaf: 2.3e-04 min_impurity_decrease: 7.28e-05	splitter: best criterion: gini max_depth: 5 min_samples_split: 9 min_samples_leaf: 14 min_weight_fraction_leaf: 1.76e-08 min_impurity_decrease: 8.42e-06	splitter: best criterion: entropy max_depth: 14 min_samples_split: 10 min_samples_leaf: 16 min_weight_fraction_leaf: 1.98e-04 min_impurity_decrease: 0.0038
RF	max_leaf_nodes: 29 n_estimators: 128 max_depth: 16 min_samples_split: 24 min_samples_leaf: 10	max_leaf_nodes: 11 n_estimators: 87 max_depth: 13 min_samples_split: 32 min_samples_leaf: 9	max_leaf_nodes: 8 n_estimators: 68 max_depth: 6 min_samples_split: 12 min_samples_leaf: 7
ET	n_estimators: 950 criterion: gini max_depth: 12 min_samples_split: 10 min_samples_leaf: 3 max_features: sqrt bootstrap: False	n_estimators: 866 criterion: entropy max_depth: 19 min_samples_split: 16 min_samples_leaf: 4 max_features: auto bootstrap: False	n_estimators: 905 criterion: log_loss max_depth: 14 min_samples_split: 20 min_samples_leaf: 5 max_features: log2 bootstrap: True
AB	n_estimators: 300 learning_rate: 0.579 algorithm: SAMME	n_estimators: 225 learning_rate: 0.310 algorithm: SAMME.R	n_estimators: 427 learning_rate: 0.110 algorithm: SAMME.R
LGBM	learning_rate: 0.007 n_estimators: 287 num_leaves: 35 max_depth: 5 min_data_in_leaf: 47	learning_rate: 0.0015 n_estimators: 929 num_leaves: 40 max_depth: 5 min_data_in_leaf: 20	learning_rate: 0.373 n_estimators: 779 num_leaves: 36 max_depth: 9 min_data_in_leaf: 34
CB	learning_rate: 0.0069 iterations: 284 depth: 8 l2_leaf_reg: 0.80908	learning_rate: 0.03349 iterations: 881 depth: 7 l2_leaf_reg: 0.2058	learning_rate: 0.0457 iterations: 536 depth: 9 l2_leaf_reg: 2.3903
XGB	booster: dart lambda: 0.004 alpha: 2.6e-04 subsample: 0.367 colsample_bytree: 0.923 early_stopping_rounds: 12 n_estimators: 16 max_depth: 7 min_child_weight: 10 eta: 0.1163 gamma: 4.57e-05 grow_policy: lossguide	booster: gbtree lambda: 2.95e-05 alpha: 2.80e-08 subsample: 0.3378 colsample_bytree: 0.586 early_stopping_rounds: 24 n_estimators: 32 max_depth: 5 min_child_weight: 7 eta: 0.7282 gamma: 0.0020 grow_policy: depthwise	booster: gbtree lambda: 1.76e-04 alpha: 1.47e-03 subsample: 0.792 colsample_bytree: 0.937 early_stopping_rounds: 22 n_estimators: 64 max_depth: 9 min_child_weight: 10 eta: 1.19e-06 gamma: 1.88e-07 grow_policy: depthwise

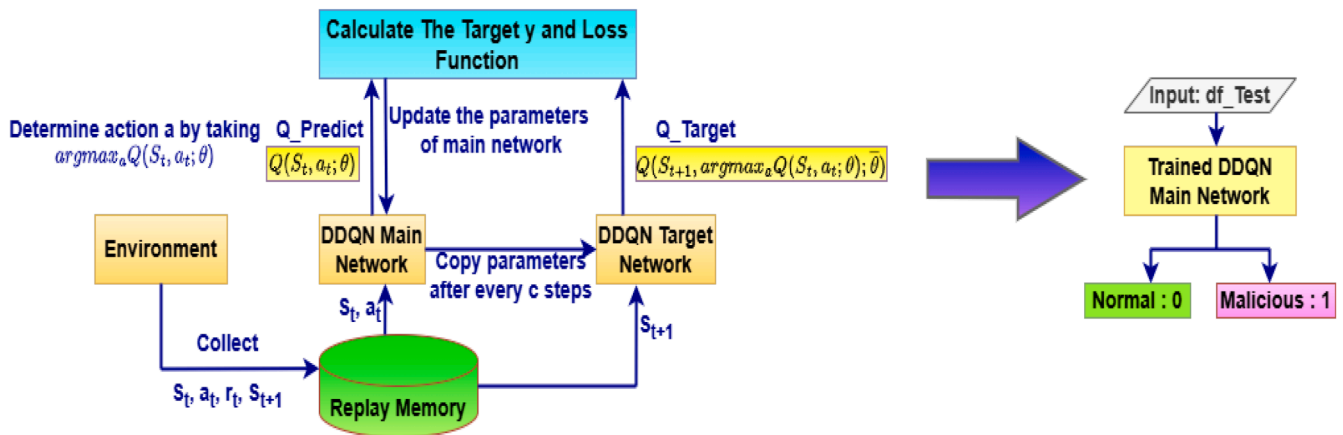


Fig. 8. Block diagram of double deep Q-network.

behavior to detect unknown DoS attacks. Moreover, the problem is designed so the agent receives feedback (rewards) during training to facilitate anomaly (or unknown DoS attack) detection in the testing phase.

A common approach involves using the DDQN model as a function

approximator to estimate Q-values associated with different class labels. The state represents the input features of the classification problem, and the actions correspond to the different class labels. If the agent accurately classifies a data sample, the environment gives positive feedback; otherwise, it provides negative feedback. The agent's prime intent is

maximizing the rewards and accurately classifying samples. The agent is a classifier (neural network model) trained using the training dataset “df_Train” as discussed in Algorithm 4. Fig. 8 illustrates the main elements of the deep reinforcement learning (DRL) framework. During training, the agent only looks at the normal traffic and known attack categories (excluding DoS attacks). Function 4 describes the training process of the double deep Q-network in detail, which takes training and testing datasets as input termed “df_Train”, and “df_Test”, respectively. The “df_Train” set is utilized during training of the double deep Q-network model, while the “df_Test” set is used only at the model prediction time. Table 14 describes the optimal hyperparameter values of the double deep Q-network model employed in this paper.

The training of the double deep Q-network involves the following steps:

- Initialize Two Networks

A primary (or main) Q-network and a target network are both neural network architectures. Here, the value (y_t) from the target network is computed using the formula provided in Eq. (38).

$$y_t = r + \gamma * Q(s_{t+1}, \operatorname{argmax}_a Q(s_{t+1}, a; \theta); \bar{\theta}) \quad (38)$$

- State (s)

The training data sample defines the environment’s “state”. The first data sample, x_1 , represents the initial state, s_1 , at the start of the training process. Each subsequent sample, x_t , corresponds to the environmental state s_t . Fig. 9 shows the layered architecture of the double deep q-network.

- Action (a)

The agent’s action is connected to the target output of the training data “df_Train”. At time step t , the action a_t taken by the agent reflects the classifier’s prediction for the data sample x_t . Since it is a binary classification problem, the action space consists of two possible actions:

Algorithm 4

Anomaly_based_IDS.

Input: Filtered Dataset (D_{f_Train} , D_{f_Test})
Output: AIDS Classification Report

procedure Anomaly_Detection_for_securing_IoT_Networks

- 1 **begin**
- 2 $df \leftarrow \text{Concatenate}(D_{f_Train}, D_{f_Test})$
- 3 **for** each ‘attack_cat’ in $df[‘attack_cat’]$:
- 4 $i \leftarrow 2$
- 5 **if** ($df[‘attack_cat’] = ‘Normal’$):
- 6 $df[‘attack_cat’] \leftarrow 0$
- 7 **end if**
- 8 **else if** ($df[‘attack_cat’] = ‘DoS’$):
- 9 $df[‘attack_cat’] \leftarrow 1$
- 10 **else if** ($df[‘attack_cat’] \neq ‘Normal’$ or $df[‘attack_cat’] \neq ‘DoS’$):
- 11 $df[‘attack_cat’] = i$
- 12 $i \leftarrow i + 1$
- 13 **end else if**
- 14 **else:**
- 15 break;
- 16 **end else**
- 17 **end for**
- 18 $df1 \leftarrow df[df[‘attack_cat’] = 1]$
- 19 $\leftarrow df1.shape[0]$
- 20 $df2 \leftarrow df[df[‘attack_cat’] \neq 1]$
- 21 $df3 \leftarrow df2[df2[‘attack_cat’] = 0].sample(m)$
- 22 $df_Train \leftarrow df2.drop(df3.index)$
- 23 $df_Test \leftarrow \text{concat}([df1, df3], \text{axis} = 0)$
- 24 AIDS Classification Report $\leftarrow \text{Double_Deep_Q_Network}(df_Train, df_Test)$
- 25 **return** AIDS Classification Report
- 26 **end**
- 27 **end procedure**

Function 4

Double_deep_Q_network(df_train, df_test).

Input: df_Train, df_Test
Output: Classification Report

procedure Double_DQN_for_Unknown_DoS_Attack_Detection

- 1 **begin**
- 2 Initialize replay memory ‘M’ to store experiences (s, a, r, s_{t+1})
- 3 Initialize main Q-network $Q(s, a; \theta)$ with random weights
- 4 Initialize target Q-network $Q(s, a; \bar{\theta})$ with $\bar{\theta} \leftarrow \theta$
- 5 Set exploration rate ‘ ϵ ’, discount factor ‘ γ ’, and learning rate ‘ τ ’
- 6 **for** each episode:
- 7 Initialize state ‘s’ // feature vector of current network traffic sample
- 8 **for** each step in episode:
- 9 With probability ‘ ϵ ’, select a random action ‘a’ (exploration)
- 10 Otherwise, select action ‘a’ that maximizes Q-value using main Q-network:
- 11 $a \leftarrow \operatorname{argmax}_a Q(s, a; \theta)$
- 12 Take action ‘a’ // classify traffic as normal or malicious
- 13 Observe reward ‘r’ and next state ‘ s_{t+1} ’ // next traffic sample’s feature vector
- 14 Store the experience (s, a, r, s_{t+1}) in replay memory ‘M’
- 15 Sample random mini-batch of experiences from ‘M’
- 16 **for** each sample (s, a, r, s_{t+1}):
- 17 Select next action $a' \leftarrow \operatorname{argmax}_a Q(s_{t+1}, a; \theta)$ using the main network
- 18 Compute target value using target network: $y \leftarrow r + \gamma * Q(s_{t+1}, a'; \bar{\theta})$
- 19 Update main Q-network by minimizing the loss: $\theta \leftarrow \theta - \tau * \nabla(Q(s, a; \theta) - y)^2$
- 20 **end for**
- 21 Periodically update target network weight: $\bar{\theta} \leftarrow \theta$
- 22 Reduce ‘ ϵ ’ to shift from exploration to exploitation
- 23 **end for**
- 24 **end for**
- 25 During testing, use the trained main Q-network to classify traffic as normal or malicious
- 26 **return** Classification Report
- 27 **end**
- 28 **end procedure**

Table 14

Hyperparameter settings of the double deep Q-network.

Hyperparameter	Values
State_size (s)	25 (UNSW-NB15), 10 (BoT-IoT), 21 (CICIoT2023)
Episode (e)	1000
Action_size (a)	2 # 0 = normal, and 1 = malicious
Gamma (γ)	0.99
Learning rate (τ)	0.001
Batch_size	64 # mini-batch size for experience replay
Epsilon (ϵ)	1 # initial exploration rate
Epsilon_min	0.01
Epsilon_decay	0.995
Memory_size (M)	10,000 # replay memory size
Target_update_freq	10 # frequency of updating target network

0 and 1, where 1 denotes the malicious class and 0 signifies the normal class, as shown in Fig. 8.

- Reward Function (r)

The reward function “r” provides feedback from the environment to assess the success or failure of the agent’s actions. Fig. 10 describes the interaction between the environment and the agent. To help the agent learn the correct classification strategy in data, the samples correctly classified by the agent neural network receive a higher reward than those incorrectly classified.

- Update Q-values

The agent updates its Q-values using the double DQN algorithm, with the target network preventing overestimating Q-values.

- Transaction Probability (P)

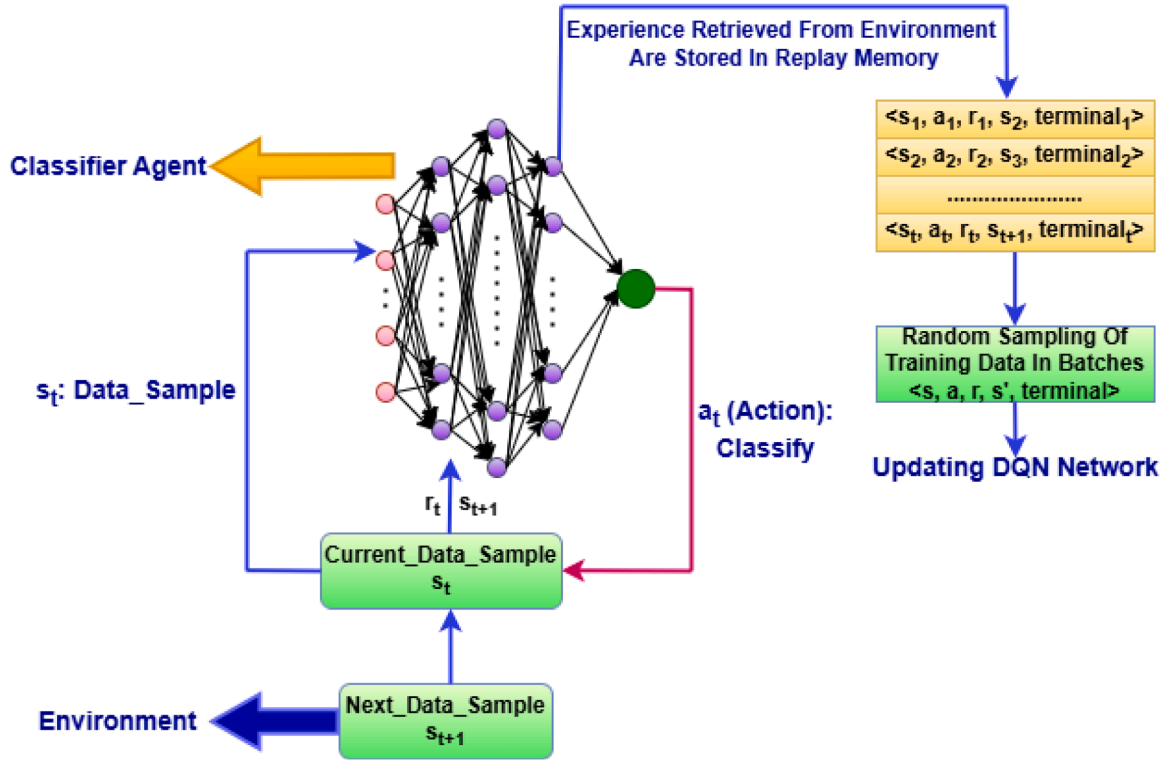


Fig. 9. Layered-architecture of double deep Q-network.

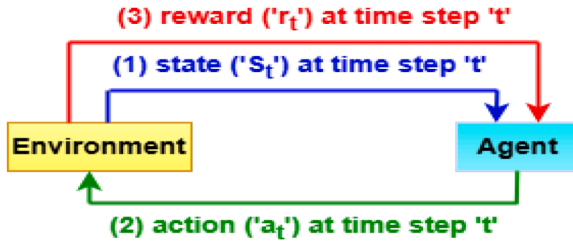


Fig. 10. Interaction between environment and agent.

The transaction probability, denoted as $P_{s_{t+1}|s_t, a_t}$, is inherently deterministic. The agent transitions from state s_t to state s_{t+1} following the sequence of samples in the training dataset.

· Discount Rate (γ)

The value of γ ranges from 0 to 1. It is utilized to balance immediate rewards and those that will be received.

· Episode (e)

An episode in reinforcement learning is the sequence of transitions $(s_0, a_0, r_0, s_1, a_1, r_1, s_2, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t)$ from the initial state to the terminal state. An episode concludes when all samples in the training dataset have been classified.

· Policy (Π_θ)

In the deep reinforcement learning framework, the policy is a classifier defined by the parameters θ . The policy decides the action a_t an agent takes at time “ t ” when in state s_t .

3.8. Evaluation metrics

Table 15 presents the evaluation metrics to assess the proposed model’s performance. It incorporates six classification metrics: “Accuracy”, “Weighted Recall”, “Weighted Precision”, “Weighted F1-Score”, “False Alarm Rate”, and “Prediction Time”.

4. Experimental results and discussion

This section outlines the experimental setup and analyzes the performance of each detection model across all three datasets using the specified evaluation metrics. Additionally, the proposed model’s performance is compared against various traditional methods and contemporary state-of-the-art approaches.

4.1. Experimental setup

All experiments in this paper are conducted using the Python programming language, leveraging libraries such as sklearn, matplotlib, keras, and tensorflow. The experiments are executed on a local server by

Table 15
Performance evaluation metrics.

Evaluation Metrics	Mathematical Formulas
Accuracy (in %)	$\frac{\sum_{i=1}^{ \text{class} } (\text{true_positive}_i)}{\text{sample}} * 100$
Weighted Precision (in %)	$\frac{\sum_{i=1}^{ \text{class} } (\text{sample}_i * \text{precision}_i)}{\text{sample}} * 100$
Weighted Recall (in %)	$\frac{\sum_{i=1}^{ \text{class} } (\text{sample}_i * \text{recall}_i)}{\text{sample}} * 100$
Weighted F1-Score (in %)	$\frac{\sum_{i=1}^{ \text{class} } (\text{sample}_i * \text{f1_score}_i)}{\text{sample}} * 100$
False Alarm Rate (FAR in %)	$\frac{\text{false_positive}_i}{\text{false_positive}_i + \text{true_negative}_i} * 100$
Prediction Time (PT in sec)	$\text{stop_time} - \text{start_time}$

running Jupyter Notebook. Additionally, Google colab is utilized to benefit from its high-speed processing and GPU capabilities. The machine used for the experiments has the following specifications: 12.0GB RAM, Intel(R) Core(TM) i5-7200 U CPU @2.50 GHz (up to 2.70 GHz), and a 64-bit operating system with an x64-based processor.

4.2. Performance analysis

Tables 16 and 17 present the multi-class classification report for the proposed SIDS and AIDS models on the UNSW-NB15 dataset. It is observed that the performance across each class is almost uniform; this can be attributed to the proposed model's effective handling of the imbalanced data issue, leading to comparable and elevated performance for each class. It is also observed that the SIDS model performs slightly better than the AIDS model.

Tables 18 and 19 present the multi-class classification report of the proposed SIDS and AIDS models, respectively, on the BoT-IoT dataset. The data indicate that the proposed SIDS model achieves remarkable performance, whereas the proposed AIDS model demonstrates comparatively lower performance than the SIDS model.

Tables 20 and 21 show the multi-class classification report of the proposed SIDS and AIDS models, respectively, on the CICIoT2023 dataset. The results for each category show notable performance, and the proposed SIDS model's accuracy is better than that of the AIDS model.

4.3. Ablation study

In this section, the performance of the proposed model is analyzed sequentially. It means that after applying each module of the proposed framework one after the other, the effectiveness of the proposed model is evaluated. Tables 22 and 23 illustrate the GAMO model's performance in generating synthetic samples. This is achieved by mixing various percentages of augmented data with the original training datasets from UNSW-NB15, BoT-IoT, and CICIoT2023, focusing on known (SIDS) and unknown (AIDS) attack detection, respectively. The synthetic samples are mixed with a 10 % difference, and the performances of the SIDS and AIDS models are evaluated. A gradual improvement in the proposed model's performance is noted. In conclusion, both tables indicate that incorporating 100 % synthetic samples into the original training dataset enhances the performance of the proposed SIDS and AIDS models across all three datasets.

Tables 24–26 show the performance of the SIDS and AIDS models after applying each phase of the proposed model sequentially. The proposed SIDS model outperforms the AIDS model primarily because SIDS utilizes the OPTUNA hyperparameter tuning technique for optimization, whereas the AIDS model's hyperparameters remain unoptimized. Additionally, the SIDS model benefits from the performance

Table 16
Performance analysis of SIDS on UNSW-NB15.

	Precision	Recall	F1-Score
Normal	0.946	0.967	0.956
Generic	0.921	0.934	0.923
Exploits	0.915	0.912	0.914
Fuzzers	0.892	0.903	0.892
DoS	0.933	0.922	0.926
Reconnaissance	0.930	0.920	0.921
Analysis	0.880	0.892	0.894
Backdoor	0.912	0.893	0.900
Shellcode	0.901	0.915	0.903
Worms	0.954	0.941	0.943
Accuracy	-	-	0.9289
Macro AVG	0.918	0.920	0.917
Weighted AVG	0.933	0.928	0.931
Train Time (sec)	64.046	-	-
Prediction Time (sec)	7.004	-	-

Table 17
Performance analysis of AIDS on UNSW-NB15.

	Precision	Recall	F1-Score
Normal	0.916	0.923	0.917
Attack	0.882	0.871	0.881
Accuracy	-	-	0.8956
Macro AVG	0.899	0.897	0.899
Weighted AVG	0.901	0.895	0.898
Train Time (Sec)	103.746	-	-
Prediction Time (Sec)	53.81	-	-

Table 18
Performance analysis of SIDS on BoT-IoT.

	Precision	Recall	F1-Score
Normal	0.992	0.991	0.992
DoS	0.994	0.983	0.989
DDoS	0.991	0.996	0.993
Reconnaissance	0.984	0.992	0.987
Information Theft	0.980	0.985	0.982
Accuracy	-	-	0.9919
Macro AVG	0.988	0.989	0.989
Weighted AVG	0.991	0.991	0.991
Train Time (Sec)	69.329	-	-
Prediction Time (Sec)	9.501	-	-

Table 19
Performance analysis of AIDS on BoT-IoT.

	Precision	Recall	F1-Score
Normal	0.905	0.908	0.907
Attack	0.912	0.910	0.910
Accuracy	-	-	0.909
Macro AVG	0.908	0.909	0.908
Weighted AVG	0.911	0.909	0.910
Train Time (Sec)	115.107	-	-
Prediction Time (Sec)	18.094	-	-

Table 20
Performance analysis of SIDS on CICIoT2023.

	Precision	Recall	F1-Score
Normal	0.985	0.988	0.986
DDoS	0.982	0.975	0.978
DoS	0.977	0.980	0.978
Recon	0.973	0.979	0.976
Web-Based	0.974	0.976	0.975
Brute Force	0.978	0.974	0.976
Spoofing	0.976	0.977	0.976
Mirai	0.979	0.978	0.978
Accuracy	-	-	0.983
Macro AVG	0.978	0.978	0.977
Weighted AVG	0.984	0.983	0.976
Train Time (Sec)	76.347	-	-
Prediction Time (Sec)	8.071	-	-

Table 21
Performance analysis of AIDS on CICIoT2023.

	Precision	Recall	F1-Score
Normal	0.945	0.948	0.946
Attack	0.952	0.949	0.950
Accuracy	-	-	0.947
Macro AVG	0.948	0.948	0.948
Weighted AVG	0.951	0.947	0.949
Train Time (Sec)	159.051	-	-
Prediction Time (Sec)	36.018	-	-

Table 22
For known attack detection (SIDS).

Dataset	Percentage of augmented samples mixing	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)
UNSW-NB15	0 %	79.19	80.40	79.19	79.85	6.65
	10 %	79.40	80.89	79.40	79.37	6.10
	20 %	78.80	80.31	78.80	79.14	7.07
	30 %	79.56	80.95	79.56	79.91	5.76
	40 %	80.36	81.66	80.36	80.78	5.19
	50 %	81.02	82.62	81.02	82.33	5.07
	60 %	81.68	86.62	81.68	82.38	4.96
	70 %	82.04	84.47	82.04	83.44	4.82
	80 %	83.26	86.32	83.26	84.54	3.93
	90 %	84.96	85.95	84.96	84.82	3.82
BoT-IoT	100 %	85.26	86.84	85.25	85.54	3.71
	0 %	85.41	86.33	85.41	85.89	7.58
	10 %	86.74	87.14	86.74	86.91	7.45
	20 %	87.99	88.38	87.99	88.11	6.90
	30 %	88.19	89.31	88.19	88.54	6.78
	40 %	89.09	89.85	89.09	89.29	6.43
	50 %	89.82	90.59	89.82	90.12	5.87
	60 %	90.46	91.42	90.46	90.71	5.62
	70 %	91.34	92.61	91.34	91.99	5.16
	80 %	92.91	93.22	92.91	93.09	4.98
CICIoT2023	90 %	93.11	94.06	93.11	93.68	4.85
	100 %	93.53	94.22	93.53	93.84	4.73
	0 %	88.17	89.07	88.17	89.90	5.02
	10 %	88.54	89.33	88.54	89.05	5.01
	20 %	88.97	89.69	88.97	89.12	4.86
	30 %	89.23	90.18	89.23	89.75	4.41
	40 %	89.66	90.72	89.66	90.18	4.15
	50 %	90.08	91.21	90.08	90.80	4.07
	60 %	90.35	91.56	90.35	90.99	3.99
	70 %	90.86	91.94	90.86	91.18	3.91
	80 %	91.17	92.23	91.17	91.19	3.86
	90 %	91.99	92.95	91.99	92.24	3.65
	100 %	92.04	93.30	92.03	92.55	3.49

Table 23
For unknown attack detection (AIDS).

Dataset	Percentage of augmented samples mixing	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)
UNSW-NB15	0 %	74.42	79.68	74.42	75.56	7.77
	10 %	74.81	79.96	74.81	75.91	7.04
	20 %	75.92	80.26	75.92	76.87	6.82
	30 %	76.87	80.89	76.87	77.37	6.64
	40 %	77.46	81.31	77.46	78.15	6.37
	50 %	78.70	81.76	78.70	79.61	5.98
	60 %	79.64	82.05	79.64	80.39	5.73
	70 %	80.61	82.62	80.61	81.48	5.24
	80 %	81.85	83.19	81.85	82.26	4.99
	90 %	82.02	83.72	82.02	82.84	4.75
BoT-IoT	100 %	83.54	84.10	83.54	83.94	4.35
	0 %	79.59	82.26	79.59	80.44	10.44
	10 %	80.37	82.67	80.37	81.60	10.02
	20 %	81.85	83.89	81.85	82.63	9.63
	30 %	82.48	84.73	82.48	83.57	9.34
	40 %	83.61	85.64	83.61	84.05	9.18
	50 %	84.09	86.11	84.09	85.45	8.73
	60 %	84.85	86.80	84.85	85.98	8.45
	70 %	85.63	87.73	85.63	86.70	8.04
	80 %	86.15	88.29	86.15	87.63	7.98
CICIoT2023	90 %	86.85	88.97	86.85	87.88	7.56
	100 %	87.09	89.39	87.09	88.18	7.19
	0 %	85.71	87.60	85.70	86.61	10.22
	10 %	85.14	87.89	85.14	86.90	10.03
	20 %	86.12	88.15	86.12	87.14	9.81
	30 %	86.87	88.71	86.87	87.60	9.36
	40 %	87.22	89.23	87.22	88.21	8.73
	50 %	88.19	90.21	88.19	89.13	7.56
	60 %	89.06	90.75	89.06	89.76	7.01
	70 %	89.50	91.32	89.50	90.04	6.84
	80 %	89.92	91.77	89.92	90.55	6.31
	90 %	90.21	92.04	90.21	91.25	5.15
	100 %	90.65	92.30	90.64	91.57	5.94

Table 24

Performance on UNSW-NB15 dataset.

Module	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
Post-applying preprocessing	SIDS	79.19	80.40	79.19	79.85	6.65	25.34
	AIDS	74.42	79.68	74.42	75.56	7.77	212.01
Post-applying data balancing	SIDS	85.26	86.84	85.25	85.54	3.71	12.921
	AIDS	83.54	84.10	83.54	83.94	4.35	111.64
Post-applying feature selection and optimization (Proposed)	SIDS	92.89	93.34	92.89	93.12	1.31	7.004
	AIDS	89.56	90.10	89.55	89.84	3.18	53.81

Table 25

Performance on BoT-IoT dataset.

Module	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
Post-applying preprocessing	SIDS	85.41	86.33	85.41	85.89	7.58	42.36
	AIDS	79.59	82.26	79.59	80.44	10.44	248.98
Post-applying data balancing	SIDS	93.53	94.22	93.53	93.84	4.73	23.024
	AIDS	87.09	89.39	87.09	88.18	7.19	88.17
Post-applying feature selection and optimization (Proposed)	SIDS	99.19	99.12	99.19	99.14	0.86	9.501
	AIDS	90.92	91.11	90.93	91.04	3.04	18.094

Table 26

Performance on CICIOT2023 dataset.

Module	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
Post-applying preprocessing	SIDS	88.17	92.07	88.17	89.90	5.02	99.848
	AIDS	85.71	87.60	85.70	86.61	10.22	160.41
Post-applying data balancing	SIDS	92.04	93.30	92.03	92.55	3.49	45.822
	AIDS	90.65	92.30	90.64	91.57	5.94	90.95
Post-applying feature selection and optimization (Proposed)	SIDS	98.31	98.40	98.30	97.61	1.18	8.071
	AIDS	94.74	95.11	94.74	94.92	2.71	36.018

advantages of tree-based models when applied to the tabular IDS dataset. The SIDS and AIDS models are evaluated across three phases: (i) after implementing the proposed framework's preprocessing module, (ii) following the integration of data preprocessing, clustering, and oversampling, and (iii) after the feature selection and optimization module. Performance improvements are seen progressively as each module is applied sequentially within the framework.

It is noted from Tables 24, 25, and 26 that the resource limitations of the IoT network are alleviated through the application of the proposed framework. The prediction time values between the last two modules of the framework, specifically (i) post-applying data balancing and (ii) post-applying feature selection and optimization, are reduced by 5.917, 13.523, and 37.751 s for the SIDS model, and 57.83, 70.076, and 54.932 s for the AIDS model on the UNSW-NB15, BoT-IoT, and CICIOT2023 datasets, respectively. The reason behind this is that the number of features is reduced for these datasets by applying a modified GWO-based feature selection technique. As a result model takes less time to predict. Moreover, the numbers indicate that the reduction in prediction time after applying the feature selection and optimization module of the proposed framework for the AIDS model is greater than that for the SIDS model. These values are derived by calculating the difference between the corresponding values of prediction time obtained after applying the feature selection and optimization module and after applying the data-balancing module.

Similarly, the minority class issue is alleviated by the introduction of the proposed GAMO module for oversampling minority class data. The values of the pair (recall, f1-score) between the first two modules ((i) Post-applying preprocessing and (ii) Post-applying data balancing) of the proposed framework increase by (6.06 %, 5.69 %), (8.12 %, 7.95 %), and (3.86 %, 2.65 %) for the SIDS model, and (9.12 %, 8.38 %), (7.5 %, 7.74 %), and (4.94 %, 4.96 %) for the AIDS model on the UNSW-NB15, BoT-IoT, and CICIOT2023 datasets, respectively. These values are derived by calculating the difference between the corresponding (recall, F1-score) pairs obtained after applying the data-balancing module and

those obtained after applying the preprocessing module.

4.4. Comparative analysis

This section assesses the proposed model's performance against traditional methods, current oversampling techniques, and cutting-edge approaches. Section 4.5 provides a critical discussion of these.

4.4.1. Comparative analysis of the proposed method with other traditional methods

Table 27 compares the proposed SIDS model with other traditional detection models, such as DT, RF, XGB, CatBoost, AdaBoost, Extra Trees, LightGBM, KNN (K-Nearest Neighbour), SVM (Support Vector Machine), DNN (Dense Neural Network), and 1D-CNN (1-dimensional CNN).

Table 28 compares the proposed oversampling technique with two other existing techniques, the synthetic minority oversampling technique (SMOTE) and the Wasserstein conditional generative adversarial network gradient penalty (WCGAN-GP).

4.4.2. Comparative analysis of the proposed method with other state-of-the-art approaches

Tables 29–32 show the proposed model's comparative analysis with contemporary state-of-the-art techniques such as (Srivastava et al., 2023; Vashishtha et al., 2023; Alghanam et al., 2023; Sharma et al., 2024; Kumar and Sinha, 2023; Tareq et al., 2022; Kumar et al., 2021; Hizal et al., 2024; Ghenni and Al-Yaseen, 2024; Srivastava and Sinha, 2024; X. Wang et al., 2024; Wang et al., 2023; Khan and Alkhathami, 2024; Abbas et al., 2024; Azimjonov and Kim, 2024; Saba et al., 2022; Beshah et al., 2024; Xing et al., 2023; Yang, 2021; Uddin et al., 2024; ElSayed et al., 2024; Jin et al., 2023; Khanday et al., 2023; Shen et al., 2024) for known and unknown attack detection.

Table 27

Comparative analysis of the proposed known attack detection model with other machine learning and deep learning-based detection models.

Dataset	Detection Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
UNSW-NB15	DT	68.14	71.46	68.14	69.41	5.83	6.72
	RF	77.18	83.90	77.18	78.63	3.71	8.56
	XGB	59.90	77.03	59.90	62.38	5.28	29.19
	CatBoost	78.60	84.97	78.60	78.78	3.59	13.64
	AdaBoost	75.55	82.54	75.55	77.36	3.98	47.14
	Extra Trees	73.01	80.17	73.01	74.55	4.23	40.68
	LightGBM	78.64	84.52	78.64	79.33	3.65	7.57
	KNN	73.38	78.23	73.38	74.77	4.37	14.82
	SVM	64.81	75.52	64.81	66.68	5.38	129.84
	DNN	72.71	77.85	72.71	73.79	4.48	10.50
	1D-CNN	71.68	81.50	71.68	73.53	4.41	16.27
	Proposed	92.89	93.34	92.89	93.12	1.31	7.004
BoT-IoT	DT	70.59	66.31	70.59	68.30	11.54	10.51
	RF	73.34	80.75	73.34	78.59	10.62	10.87
	XGB	34.18	11.96	34.18	17.55	19.00	12.93
	CatBoost	76.39	82.26	76.39	77.34	9.57	15.32
	AdaBoost	72.69	68.83	72.69	69.30	10.66	23.80
	Extra Trees	74.02	68.42	74.02	69.91	10.31	18.69
	LightGBM	76.31	82.34	76.31	78.98	9.59	9.75
	KNN	75.90	75.24	75.90	75.27	9.60	20.35
	SVM	73.07	75.43	73.07	74.22	9.93	115.92
	DNN	73.58	67.83	73.58	69.74	10.11	36.86
	1D-CNN	73.53	76.76	73.53	74.67	10.43	81.93
	Proposed	99.19	99.12	99.19	99.14	0.86	9.501
CICIoT 2023	DT	88.38	92.16	88.38	89.23	2.99	10.577
	RF	88.88	90.95	88.88	89.22	3.80	8.550
	XGB	89.11	92.41	89.11	90.14	2.69	9.399
	CatBoost	89.73	91.76	89.73	90.37	2.49	11.972
	AdaBoost	90.12	93.64	90.11	90.85	2.73	67.490
	Extra Trees	90.60	91.73	90.59	90.96	3.00	61.220
	LightGBM	89.65	92.90	89.65	90.29	2.99	8.920
	KNN	88.69	89.81	88.69	89.05	3.81	66.73
	SVM	88.53	91.75	88.53	90.48	3.20	273.62
	DNN	88.64	92.76	88.63	90.18	3.70	9.823
	1D-CNN	89.15	93.32	89.14	90.09	3.62	8.813
	Proposed	98.31	98.40	98.30	97.61	1.18	8.071

Table 28

Comparative analysis with other oversampling techniques.

Dataset	Oversampling Technique	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
SIDS	UNSW-NB15	None	77.57	80.54	77.57	74.90	7.78
		SMOTE	75.88	80.65	75.87	72.37	6.93
		WCGAN-GP	80.08	82.72	80.08	77.18	5.70
		Modified GAMO	92.89	93.34	92.89	93.12	1.31
	BoT-IoT	None	79.83	80.44	79.82	79.23	6.99
		SMOTE	65.68	80.82	65.67	66.77	6.57
		WCGAN-GP	87.79	87.16	87.79	87.04	4.83
		Modified GAMO	99.19	99.12	99.19	99.14	0.86
	CICIoT2023	None	89.13	89.48	89.13	89.24	2.82
		SMOTE	91.21	83.77	91.20	87.77	3.65
		WCGAN-GP	94.14	94.84	94.14	94.36	1.52
		Modified GAMO	98.31	98.40	98.30	97.61	1.18
AIDS	UNSW-NB15	None	69.25	63.23	69.25	64.56	10.31
		SMOTE	73.20	75.70	73.20	68.72	9.92
		WCGAN-GP	81.41	81.32	81.41	79.43	5.10
		Modified GAMO	89.56	90.10	89.55	89.84	3.18
	BoT-IoT	None	42.51	54.02	42.50	40.08	7.99
		SMOTE	67.14	79.90	67.13	67.59	3.52
		WCGAN-GP	85.68	85.21	85.68	85.40	3.30
		Modified GAMO	90.92	91.11	90.93	91.04	3.04
	CICIoT2023	None	70.73	81.34	70.72	71.95	3.12
		SMOTE	88.54	89.00	88.53	88.56	5.28
		WCGAN-GP	92.36	93.27	92.36	93.71	3.95
		Modified GAMO	94.74	95.11	94.74	94.92	2.71

4.5. Discussion

This section elaborates on the critical discussions found in [Section 4.4](#), organized into three main subsections that reflect the research objectives outlined in this paper: resource efficiency, imbalanced IDS dataset, and detection accuracy.

4.5.1. Resource efficiency

One method of achieving resource efficiency in an IoT environment is to design a lightweight model. This can be accomplished by designing the model on data with a reduced number of features or lower dimensionality ([Srivastava and Sinha, 2024](#)). This section discusses the various dimensionality reduction techniques employed in the state-of-the-art

Table 29

Comparison of the proposed model with (Sharma et al., 2024; Kumar et al., 2021) on the UNSW-NB15 dataset.

Category	(Sharma et al., 2024)			(Kumar et al., 2021)		Proposed		
	Precision	Recall	F1-Score	Precision	Recall	Precision	Recall	F1-Score
DoS	0.57	0.04	0.07	0.7883	0.2946	0.93	0.92	0.92
Exploits	0.69	0.91	0.78	0.7374	0.8569	0.91	0.91	0.91
Fuzzers	0.60	0.80	0.69	-	-	0.89	0.90	0.89
Generic	1.00	0.97	0.99	0.9866	0.9858	0.92	0.93	0.92
Normal	0.91	0.81	0.86	0.9284	0.9619	0.94	0.96	0.95
Accuracy	0.81			0.8892		0.9289		

Table 30

Comparison of the proposed model with (Tareq et al., 2022; Yang, 2021) on the UNSW-NB15 dataset.

Category	(Tareq et al., 2022)			(Yang, 2021)		Proposed		
	Precision	Recall	F1-Score	Precision	F1-Score	Precision	Recall	F1-Score
Normal	1.00	1.00	1.00	0.97	0.85	0.94	0.96	0.95
Generic	0.99	0.98	0.99	0.99	0.99	0.92	0.93	0.92
Exploits	0.58	0.93	0.72	0.88	0.94	0.91	0.91	0.91
Fuzzers	0.67	0.54	0.60	0.34	0.48	0.89	0.90	0.89
DoS	0.51	0.06	0.11	0.97	0.93	0.93	0.92	0.92
Reconnaissance	0.91	0.73	0.81	0.89	0.94	0.93	0.92	0.92
Analysis	0.42	0.05	0.09	0.00	0.00	0.88	0.89	0.89
Backdoor	0.69	0.07	0.13	1.00	1.00	0.91	0.89	0.90
Shellcode	0.55	0.58	0.57	0.59	0.74	0.90	0.91	0.90
Worms	0.80	0.11	0.20	0.83	0.83	0.95	0.94	0.94
Accuracy	0.984			0.772		0.928		

Table 31

Comparative analysis of proposed known attack detection (SIDS) model with state-of-the-art.

Dataset	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
UNSW-NB15	(Srivastava et al., 2023)	86.52	84.03	86.51	84.80	1.51	-
	(Vashishtha et al., 2023)	92.7	95.1	92.7	93.5	-	-
	(Alghanam et al., 2023)	94.45	-	-	91.35	-	-
	(Kumar and Sinha, 2023)	-	81.39	81.54	81.46	12.50	-
	(Tareq et al., 2022)	98.4	99.0	97.9	98.5	-	-
	(Srivastava and Sinha, 2024)	90.63	92.27	90.63	91.44	1.02	13.81
	(X. Wang et al., 2024)	90.35	77.31	74.42	74.58	-	-
	(Yang, 2021)	77.27	83.99	-	90.51	22.92	-
	Proposed	92.89	93.34	92.89	93.12	1.31	7.004
	(Alghanam et al., 2023)	97.37	-	-	94.88	-	-
BoT-IoT	(Kumar and Sinha, 2023)	-	99.42	99.16	99.29	1.33	-
	(Azimjonov and Kim, 2024)	92.48	92.28	92.48	92.37	-	0.00015
	(Saba et al., 2022)	92.85	-	-	-	-	-
	Proposed	99.19	99.12	99.19	99.14	0.86	9.501
	(Hizal et al., 2024)	91.27	91.45	94.30	92.32	0.009	59.83
CICIoT2023	(Gheni and Al-Yaseen, 2024)	97.46	97	97	97	-	0.03
	(X. Wang et al., 2024)	97.44	89.10	87.47	87.73	-	-
	(Wang et al., 2023)	93.13	91.80	93.13	91.94	-	6.4
	(Khan and Alkhathami, 2024)	95.54	95.55	95.54	95.51	-	6.64
	(Abbas et al., 2024)	98.61	98.55	98.61	98.57	-	-
	Proposed	98.31	98.40	98.30	97.61	1.18	8.071

Table 32

Comparative analysis of proposed unknown attack detection model (AIDS) with state-of-the-art.

Dataset	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FAR (%)	Prediction Time (sec)
UNSW-NB15	DUA-IDS (Xing et al., 2023)	88.47	87.94	-	88.54	-	-
	usfAD (Uddin et al., 2024)	82.15	97.22	71.55	82.43	-	-
	DQN-HIDS (Shen et al., 2024)	86.17	85.69	61.85	71.85	-	-
	Proposed	89.56	90.10	89.55	89.84	3.18	53.81
BoT-IoT	EIDS (Jin et al., 2023)	86.7	81	72	-	-	-
	Naïve Bayes (Khanday et al., 2023)	88	90.5	88.5	93	-	-
	Linear SVC (Khanday et al., 2023)	88	90.5	88	87.5	-	-
	Proposed	90.92	91.11	90.93	91.04	3.04	18.094
CICIoT2023	AUWPAE (Beshah et al., 2024)	99.33	99.88	96.53	97.78	-	-
	(ElSayed et al., 2024)	93.6	-	-	-	-	-
	Proposed	94.74	95.11	94.74	94.92	2.71	36.018

intrusion detection system (IDS) domain. Table 31 presents several metaheuristic algorithms, such as (Srivastava and Sinha, 2024), which employs PSO and ACO techniques for feature selection in two phases, while in (Srivastava et al., 2023), the genetic algorithm is used to identify the significant features. Pigeon-inspired optimization and local search (LS-PIO) for selecting features in IoT network traffic data aimed at signature-based intrusion detection (Alghanam et al., 2023). However, the correlated features in the dataset remain intact because the LS-PIO algorithm chooses features according to the feature subset rate, FPR, and TPR. In contrast, the proposed model focuses on selecting the most effective features that are uncorrelated, achieving high detection accuracy, low false alarm rate (FAR), and a reduced rate of feature subsets.

In (Gheni and Al-Yaseen, 2024), intrusion detection employs feature selection based on information gain, follows a two-step data clustering approach, and classifies using both an autoencoder and a multi-layer perceptron. Additionally, correlated features are not reduced in this process. Features in (Kumar et al., 2021) are selected using information gain but do not remove correlated features. A lightweight IDS model (Wang et al., 2023) for an IoT network is designed, which reduces data dimensionality using incremental principal component analysis. However, the incremental PCA-based dimensionality reduction technique is computationally expensive. In (Khan and Alkhathami, 2024), features are reduced using Pearson correlation. The Pearson correlation coefficient provides a measure of linear relationship between the two features and can fail to capture nonlinear relationships. In (Azimjonov and Kim, 2024), a lightweight Intrusion Detection System (IDS) uses a Stochastic Gradient Descent Classifier (SGDC) with feature selection via importance coefficient, correlation coefficient, and backward and forward sequential methods. The framework (Kumar and Sinha, 2023) employs Autoencoder for dimensionality reduction. Autoencoders involve training deep neural networks, which can be computationally intensive. Spatial features in (X. Wang et al., 2024) are extracted with convolutional and residual units, while temporal features are extracted utilizing the BiGRU model.

Most of the existing approaches lack selecting uncorrelated and important features based on detection metrics as well as the resource constraint metrics. In contrast, this paper selects relevant features using a modified GWO-based feature selection technique based on multiple objectives: uncorrelated features, smaller subsets, lower error rates, fewer false alarms, and higher accuracy. The proposed model enhances the efficiency of resources in IoT networks by reducing the prediction time and gives better results compared to the aforementioned approaches.

4.5.2. Imbalanced IDS dataset

In this subsection, several state-of-the-art methods are discussed that address the imbalanced IDS dataset using techniques like SMOTE, tabular GAN, Wasserstein GAN, and Wasserstein GAN-GP. However, some of the state-of-the-art methods do not address the imbalanced data problem. Table 28 shows that the modified GAMO-based oversampling technique outperforms the other two methods. SMOTE-based oversampling is inefficient due to producing redundant, noisy samples and lacks diversity while struggling with complex distributions and overfitting. Although WCGAN-GP improves on SMOTE (Srivastava et al., 2023) by addressing mode collapse and vanishing gradient issues, it requires careful tuning of hyperparameters like the gradient penalty coefficient (γ). After extensive training, it may exhibit unstable behavior in the generator's loss function.

Table 29 compares the proposed model's multi-class classification results with (Sharma et al., 2024; Kumar et al., 2021) on the UNSW-NB15 dataset across five categories: DoS, Exploits, Fuzzers, Generic, and Normal. In (Sharma et al., 2024), deep learning models like DNN and CNN are used, but they do not address the minority class problem. Performance is notably lower for the DoS category, with overall accuracy inferior to the proposed model. The UIDS model

(Kumar et al., 2021) employs decision tree algorithms (C5, CHAID, QUEST, CART) on the same dataset, deriving a rule set from each trained model without addressing the minority class issue. In (Tareq et al., 2022), two deep learning models, DenseNet121 and Inception Time, adapted for one-dimensional UNSW-NB15 input data, do not address data imbalance. They perform well in classes like Normal and Generic but poorly in Exploits, Fuzzers, DoS, and Analysis, as shown in Table 30. This issue stems from training on an imbalanced dataset, resulting in misleading results favoring abundant sample classes. EnsembleNet (Yang, 2021), with multiple networks like CNNs and RNNs, performs adequately for several classes but struggles with Fuzzers, Analysis, and Shellcode, highlighting persistent data imbalance issues—high accuracy for majority classes but low for minority ones. The proposed model surpasses those from (Tareq et al., 2022; Yang, 2021), showing balanced performance across nearly all classes, generating synthetic data that resembles real data and leading to fair detection accuracy.

In Table 31, the data imbalance issue remains unaddressed in (Alghanam et al., 2023; Gheni and Al-Yaseen, 2024; Srivastava and Sinha, 2024; Wang et al., 2023; Abbas et al., 2024; Saba et al., 2022). Wasserstein conditional GAN (Srivastava et al., 2023) with gradient penalty (WCGAN-GP) tackles class imbalance in the dataset. The hybrid model (HIDM) in (Vashishtha et al., 2023) integrates SDM and ADM using majority voting, but it fails to address the data imbalance issue. The framework (Kumar and Sinha, 2023) employs WCGAN for creating synthetic attack samples. The data imbalance issue in (X. Wang et al., 2024) is addressed with a conditional tabular GAN. Random undersampling and SMOTE-based oversampling are used to address this issue in (Khan and Alkhathami, 2024). However, data balancing through the random undersampling technique can lead to the loss of some important information. In Table 32, a technique detects DoS attacks in IoT networks (Khanday et al., 2023). It uses feature selection with the Extra Tree classifier to assess importance and SMOTE for class imbalance.

In contrast, the proposed model effectively handles the minority class problem with the attention-based enhanced generative modeling approach. An innovative end-to-end oversampling method, GAMO (Mullick et al., 2019), is introduced here, which uses the convex hull concept to constrain synthetic samples, yielding a distribution similar to real samples. The modified GAMO model enhances this with an attention mechanism at the initial layer, focusing on key features of the real data distribution to generate realistic synthetic samples. Table 28 shows that the proposed modified GAMO model is superior to other existing techniques, such as SMOTE and WCGAN-GP. This effectively mitigates class imbalance, thereby enhancing performance through augmented training datasets. The proposed oversampling consistently outperforms other techniques.

4.5.3. Detection accuracy

In this subsection, several state-of-the-art methods are discussed that improve the detection accuracy of the IDS model through advanced detection techniques and hyperparameter optimization of models. Table 27 results utilize all model modules except the signature-based IDS (SIDS). Different detection models are applied sequentially to evaluate their performance. In Table 31, the hybrid model (HIDM) in (Vashishtha et al., 2023) integrates SDM and ADM using majority voting, but it does not optimize hyperparameters. Various one-class classification models in (Alghanam et al., 2023), including isolation forest and one-class SVM, are combined in a voting classifier to detect attacks, yet classification algorithm optimization remains unaddressed. A genetic algorithm in (Srivastava and Sinha, 2024) optimizes hyperparameters for various machine learning and deep learning models. In (Wang et al., 2023), an OPTUNA-optimized deep neural network with the Bi-LSTM model classifies the data. Dynamic quantization trims the classification model's structure. In (Khan and Alkhathami, 2024), the model performance is evaluated using random forest, logistic regression, adaptive boosting, perceptron, and deep neural networks. Optimization of the models is not considered. In (Azimjonov and Kim, 2024), it uses

the grid-search approach for hyperparameter tuning, which is computationally inefficient. The approach in (Hizal et al., 2024) introduces a two-stage DDoS attack detection model tailored for IoT, surpassing DNN, CNN, RNN, and LSTM, and executed on a partially balanced dataset without optimizing the model. The framework (Kumar and Sinha, 2023) employs Xgboost for classification; however, it does not take into account the model optimization.

In (Srivastava et al., 2023), multiple machine-learning models are used to design the IDS for attack detection, which is not optimized. In contrast, this paper employs a majority voting classifier to aggregate predictions of optimized classifiers, improving performance. The hyperparameter optimization of the deep learning model is ignored in (X. Wang et al., 2024). In (Abbas et al., 2024), DNN, CNN, and RNN models predict attack and normal classes. It uses grid-search-based hyperparameter tuning, which is computationally inefficient. In (Saba et al., 2022), a CNN-based approach addresses anomaly detection in IoT, but it does not handle hyperparameter optimization.

In (Xing et al., 2023), a dynamic unknown attack IDS (DUA-IDS) is proposed, incorporating three main components. A convolutional neural network and transformer model extract features from the data in (Xing et al., 2023). Threshold-based classification identifies known and unknown attacks. The final module learns network traffic behavior, filtering existing knowledge while adapting to new attacks. As shown in Table 32, the performance in (Xing et al., 2023) is significantly lower due to inadequate learning of network traffic behavior. Conversely, the double-deep Q network in the proposed unknown DoS attack detection module adapts better to dynamic traffic behavior, learns optimal decision-making policies, and, importantly, reduces false positives over time. The authors in (ElSayed et al., 2024) have implemented an autoencoder to enhance healthcare security in the IoT framework. It detects zero-day attacks by calculating the mean squared error between the decoder's output and the original input. If the error exceeds a threshold, the input is marked as malicious. A random search method optimizes the autoencoder's hyperparameters, though it can miss the best combinations due to its reliance on random selection.

Most existing approaches, such as (Srivastava et al., 2023; Vashishtha et al., 2023; Alghanam et al., 2023; Kumar and Sinha, 2023; Hizal et al., 2024; Ghani and Al-Yaseen, 2024; X. Wang et al., 2024; Khan and Alkhatami, 2024; Saba et al., 2022), do not consider hyperparameter optimization, which significantly degrades model performance. Some of the models consider grid-search or random search-based approaches. However, grid search-based hyperparameter tuning is computationally very inefficient. Furthermore, because of the random nature of the random search-based hyperparameter tuning, it may not consistently identify the global optimal set of hyperparameters. In contrast, the proposed SIDS model outperforms other detection models by using a majority voting classifier that integrates seven tree-based optimized (optimized using OPTUNA-based hyperparameter tuning) algorithms: Decision Tree, Random Forest, Extra Tree, AdaBoost, LightGBM, CatBoost, and Xgboost. This improves detection performance by focusing on key dataset features. This integration enhances accuracy and efficiency, leveraging the strengths of all base models for quick, interpretable, and robust predictions while reducing bias and variation. Tree-based models are preferred for decision-making in intrusion detection, as they outperform deep neural networks in anomaly detection for tabular data (Zoppi et al., 2024). It is scrutinized that the proposed model performs better than other existing approaches in most cases. This demonstrates the effectiveness of the proposed “ARLHNIDS-IoT” model.

5. Conclusion

The proposed “ARLHNIDS-IoT” works as a watchdog in the IoT network framework to secure IoT devices from known and unknown cyber-attacks. This paper removes the imbalanced data problem using the generative model termed the modified generative adversarial minority oversampling technique, which integrates the attention

mechanism. A modified grey wolf optimization-based feature selection technique is utilized for effective feature selection. It selects the uncorrelated and highly effective features that enhance the proposed model's detection performance. This paper uses an ensemble of seven tree-based intelligent detection models for known attack detection. These models are ensembled using the majority voting classifier. The hyperparameters of each tree-based detection model are optimized using the OPTUNA-based hyperparameter tuning technique. The known attack detection provides a multi-class classification report, classifying between normal and malicious traffic and detecting different malicious traffic categories. This paper uses the double deep Q-network for unknown attack detection, which gives binary classification reports indicating either normal or malicious behavior. It detects the unknown DoS attack. Extensive experiments are performed to scrutinize the effectiveness of the proposed SIDS and AIDS model on three IoT-based IDS datasets: UNSW-NB15, BoT-IoT, and CICIoT2023. The proposed model is compared with several current state-of-the-art techniques, and it is observed to outperform most other techniques in terms of accuracy, precision, recall, F1-score, false alarm rate (FAR), and prediction time. The proposed oversampling technique is compared with existing ones, such as SMOTE and WCGAN-GP. It is also observed from the detection results that the proposed detection model achieves higher performance than other techniques. The performance of the proposed SIDS model is compared with other machine learning and deep learning-based individual detection models, such as DT, RF, XGB, CatBoost, AdaBoost, Extra Trees, LightGBM, KNN, SVM, DNN, and 1D-CNN. From the detection result, it is noted that the proposed SIDS model outperforms others. The main limitation of the paper is that it does not explore any real-time IoT-based traffic. Moreover, this paper does not optimize the hyperparameters of the double deep Q-network. Future directions involve extending the research to real-time datasets and investigating novel and improved techniques for hyperparameter optimization in both SIDS and AIDS models to enhance the attack detection rate and reduce FAR.

CRedit authorship contribution statement

Arpita Srivastava: Writing – original draft, Visualization, Validation, Methodology, Data curation, Conceptualization. **Ditipriya Sinha:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

There is no conflict of interest.

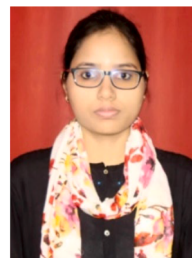
Data availability

Data will be made available on request.

References

- Abbas, S., Alsubai, S., Ojo, S., Sampedro, G.A., Almadhor, A., Hejaili, A.A., Bouazzi, I., 2024. An efficient deep recurrent neural network for detection of cyberattacks in realistic IoT environment. *J. Supercomput.* 1–19.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: a next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631.
- Alghanam, O.A., Almobaideen, W., Saadeh, M., Adwan, O., 2023. An improved PIO feature selection algorithm for IoT network intrusion detection system based on ensemble learning. *Expert. Syst. Appl.* 213, 118745.
- Alghayadh, F., Debnath, D., 2020a. Hid-smart: hybrid intrusion detection model for smart home. In: *2020 10th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, pp. 0384–0389.
- Alghayadh, F., Debnath, D., 2020b. A hybrid intrusion detection system for smart home security. In: *2020 IEEE International Conference on Electro Information Technology (EIT)*. IEEE, pp. 319–323.
- Aydin, M.A., Zaim, A.H., Ceylan, K.G., 2009. A hybrid intrusion detection system design for computer network security. *Comput. Electr. Eng.* 35 (3), 517–526.

- Azimjonov, J., Kim, T., 2024. Stochastic gradient descent classifier-based lightweight intrusion detection systems using the efficient feature subsets of datasets. *Expert. Syst. Appl.* 237, 121493.
- Beshah, Y.K., Abebe, S.L., Melaku, H.M., 2024. Drift adaptive online DDoS attack detection framework for IoT system. *Electron. (Basel)* 13 (6), 1004.
- Chohra, A., Shirani, P., Karbab, E.B., Debbabi, M., 2022. Chameleon: optimized feature selection using particle swarm optimization and ensemble methods for network anomaly detection. *Comput. Secur.* 117, 102684.
- Chowdhury, R., Sen, S., Goswami, A., Purkait, S., Saha, B., 2023. An implementation of bi-phase network intrusion detection system by using real-time traffic analysis. *Expert. Syst. Appl.* 224, 119831.
- ElSayed, Z., Elsayed, N., Bay, S., 2024. A novel zero-trust machine learning green architecture for Healthcare IoT cybersecurity: review, analysis, and implementation. *Southeast. Con.* 2024, 686–692.
- Gheni, H.Q., Al-Yaseen, W.L., 2024. Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset. *E-prime-advances in Electrical engineering. Electron. Energy* 9, 100673.
- Haroon, A., Shah, M.A., Asim, Y., Naeem, W., Kamran, M., Javaid, Q., 2016. Constraints in the IoT: the world in 2020 and beyond. *Int. J. Adv. Comput. Sci. Appl.* 7 (11), 37 (1), 125–132.
- Hizal, S., Cavusoglu, U., Akgun, D., 2024. A novel deep learning-based intrusion detection system for IoT DDoS security. *Internet Things* 28, 101336.
- Jin, D., Chen, S., He, H., Jiang, X., Cheng, S., Yang, J., 2023. Federated incremental learning based evolvable intrusion detection system for zero-day attacks. *IEEE Netw.* 37 (1), 125–132.
- Kanna, P.R., Santhi, P., 2022. Hybrid intrusion detection using mapreduce based black widow optimized convolutional long short-term memory neural networks. *Expert. Syst. Appl.* 194, 116545.
- Khan, M.M., Alkhatami, M., 2024. Anomaly detection in IoT-based healthcare: machine learning for enhanced security. *Sci. Rep.* 14 (1), 5872.
- Khanday, S.A., Fatima, H., Rakesh, N., 2023. Implementation of intrusion detection model for DDoS attacks in Lightweight IoT Networks. *Expert. Syst. Appl.* 215, 119330.
- Khrasat, A., Gondal, I., Vamplew, P., Kamruzzaman, J., Alazab, A., 2019. A novel ensemble of hybrid intrusion detection system for detecting internet of things attacks. *Electronics. (Basel)* 8 (11), 1210.
- Kim, G., Lee, S., Kim, S., 2014. A novel hybrid intrusion detection method integrating anomaly detection with misuse detection. *Expert. Syst. Appl.* 41 (4), 1690–1700.
- Koroniotis, N., Moustafa, N., Sitnikova, E., Turnbull, B., 2019. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: bot-iot dataset. *Future Gener. Comput. Syst.* 100, 779–796.
- Kumar, V., Das, A.K., Sinha, D., 2021. UIDS: a unified intrusion detection system for IoT environment. *Evol. Intell.* 14 (1), 47–59.
- Kumar, V., Sinha, D., 2023. Synthetic attack data generation model applying generative adversarial network for intrusion detection. *Comput. Secur.* 125, 103054.
- Li, S., Cao, Y., Liu, S., Lai, Y., Zhu, Y., Ahmad, N., 2024. Hda-ids: a hybrid dos attacks intrusion detection system for iot by using semi-supervised cl-gan. *Expert. Syst. Appl.* 238, 122198.
- Mirjalili, S., Mirjalili, S.M., Lewis, A., 2014. Grey wolf optimizer. *Adv. Eng. Softw.* 69, 46–61.
- Moustafa, N., Slay, J., 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). IEEE, pp. 1–6.
- Mullick, S.S., Datta, S., Das, S., 2019. Generative adversarial minority oversampling. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1695–1704.
- Nandanwar, H., Katarya, R., 2024. Deep learning enabled intrusion detection system for industrial IOT environment. *Expert. Syst. Appl.* 249, 123808.
- Nazir, A., Khan, R.A., 2021. A novel combinatorial optimization based feature selection method for network intrusion detection. *Comput. Secur.* 102, 102164.
- Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A., 2023. CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 23 (13), 5941.
- Nguyen, X.H., Le, K.H., 2023. Robust detection of unknown DoS/DDoS attacks in IoT networks using a hybrid learning model. *Int. Things* 23, 100851.
- Ozkan-Okay, M., Aslan, Ö., Eryigit, R., Samet, R., 2021. SABADT: hybrid intrusion detection approach for cyber attacks identification in WLAN. *IEEE Access* 9, 157639–157653.
- Papamartzivanos, D., Mármol, F.G., Kambourakis, G., 2018. Dendron: genetic trees driven rule induction for network intrusion detection systems. *Future Gener. Comput. Syst.* 79, 558–574.
- Peterson, J.M., Leevy, J.L., Khoshgoftaar, T.M., 2021. A review and analysis of the bot-iot dataset. In: 2021 IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE, pp. 20–27.
- Rahman, S., Pal, S., Mittal, S., Chawla, T., Karmakar, C., 2024. SYN-GAN: a robust intrusion detection system using GAN-based synthetic data for IoT security. *Int. Things* 26, 101212.
- Rizzardi, A., Sicari, S., Porisini, A.C., 2024. NERO: nEural algorithmic reasoning for zeRO-day attack detection in the IoT: a hybrid approach. *Comput. Secur.* 142, 103898.
- Saba, T., Rehman, A., Sadad, T., Kolivand, H., Bahaj, S.A., 2022. Anomaly-based intrusion detection system for IoT networks through deep learning model. *Comput. Electr. Eng.* 99, 107810.
- Sharma, B., Sharma, L., Lal, C., Roy, S., 2024. Explainable artificial intelligence for intrusion detection in IoT networks: a deep learning based approach. *Expert. Syst. Appl.* 238, 121751.
- Shen, S., Cai, C., Li, Z., Shen, Y., Wu, G., Yu, S., 2024. Deep Q-network-based heuristic intrusion detection against edge-based SIoT zero-day attacks. *Appl. Soft. Comput.* 150, 111080.
- Simon, J., Kapileswar, N., Polasi, P.K., Elaveini, M.A., 2022. Hybrid intrusion detection system for wireless IoT networks using deep learning algorithm. *Comput. Electr. Eng.* 102, 108190.
- Srivastava, A., Sinha, D., 2024. PSO-ACO-based bi-phase lightweight intrusion detection system combined with GA optimized ensemble classifiers. *Clust. Comput.* 27 (10), 14835–14890.
- Srivastava, A., Sinha, D., 2025. FP-growth-based signature extraction and unknown variants of DoS/DDoS attack detection on real-time data stream. *J. Inf. Secur. Appl.* 89, 103996.
- Srivastava, A., Sinha, D., Kumar, V., 2023. WCGAN-GP based synthetic attack data generation with GA based feature selection for IDS. *Comput. Secur.* 134, 103432.
- Tareq, I., Elbagoury, B.M., El-Regaily, S., El-Horbaty, E.S.M., 2022. Analysis of ton-iot, unw-nb15, and edge-iiot datasets using dl in cybersecurity for iot. *Appl. Sci.* 12 (19), 9572.
- Touré, A., Imine, Y., Semnont, A., Delot, T., Gallais, A., 2024. A framework for detecting zero-day exploits in network flows. *Comput. Netw.* 248, 110476.
- Uddin, M.A., Aryal, S., Bouadjene, M.R., Al-Hawawreh, M., & Talukder, M.A. (2024). usfAD based effective unknown attack detection focused IDS framework. *arXiv preprint arXiv:2403.11180*.
- Vashishtha, L.K., Singh, A.P., Chatterjee, K., 2023. HIDM: a hybrid intrusion detection model for cloud based systems. *Wirel. Pers. Commun.* 128 (4), 2637–2666.
- Wang, X., Dai, L., Yang, G., 2024a. A network intrusion detection system based on deep learning in the IoT. *J. Supercomput.* 80 (16), 24520–24558.
- Wang, Z., Chen, H., Yang, S., Luo, X., Li, D., Wang, J., 2023. A lightweight intrusion detection method for IoT based on deep learning and dynamic quantization. *PeerJ Comput. Sci.* 9, e1569.
- Wang, Z., Li, J., Yang, S., Luo, X., Li, D., Mahmoodi, S., 2024b. A lightweight IoT intrusion detection model based on improved BERT-of-Theseus. *Expert. Syst. Appl.* 238, 122045.
- Wu, Y., Hu, Y., Wang, J., Feng, M., Dong, A., Yang, Y., 2024. An active learning framework using deep Q-network for zero-day attack detection. *Comput. Secur.* 139, 103713.
- Xing, N., Zhao, S., Wang, Y., Ning, K., Liu, X., 2023. A dynamic intrusion detection system capable of detecting unknown attacks. *Int. J. Adv. Comput. Sci. Appl.* 14 (7).
- Yang, L., Moubayed, A., Shami, A., 2021. MTH-IDS: a multitiered hybrid intrusion detection system for internet of vehicles. *IEEE Internet. Things. J.* 9 (1), 616–632.
- Yang, S., 2021. Deep Neural Networks for Network Intrusion Detection.
- Yu, S., Zhai, R., Shen, Y., Wu, G., Zhang, H., Yu, S., Shen, S., 2023. Deep q-network-based open-set intrusion detection solution for industrial internet of things. *IEEE Internet. Things. J.*
- Zhang, L., Ma, D., 2022. A hybrid approach toward efficient and accurate intrusion detection for in-vehicle networks. *IEEE Access* 10, 10852–10866.
- Zoppi, T., Gazzini, S., Ceccarelli, A., 2024. Anomaly-based error and intrusion detection in tabular data: no DNN outperforms tree-based classifiers. *Future Gener. Comput. Syst.* 160, 951–965.



Arpita Srivastava is pursuing a Ph.D. degree in the Department of Computer Science and Engineering from the National Institute of Technology Patna, Bihar, India. She has received her Master of Technology degree in the Department of Computer Science and Engineering from Kamla Nehru Institute of Technology, Sultanpur, Uttar Pradesh, India. Her research interests include Intrusion Detection Systems, Machine Learning, and Deep Learning.



Ditipriya Sinha has received PhD degree in the Department of Computer Science and Technology, Indian Institute of Engineering Science and Technology (IIST), Shibpur. She is the silver medalist during M.Tech. She is presently serving as an Assistant Professor (Grade I) in the Department of Computer Science and Engineering, National Institute of Technology Patna. Previously, She was an Assistant Professor in the Department of Computer Science and Engineering at Birla Institute of Technology, Mesra. She has over 60 publications, including many reputed high-impact factor SCI-indexed journals. She has completed one DST SERB project on “Energy EFFICIENT RELIABLE DATA DELIVERY during FOREST FIRE using WIRELESS NETWORK”. Her 3 patents are granted, 2 of which are Indian and 1 German patent. She is one of the editors of the edited book titled “The Role of Blockchain in Disaster Management” which is published in Elsevier. Her another edited book entitled “Emerging Trends in Artificial Intelligence Based IoT: Techniques, Applications and Security” is at the copy editing stage by Bentham publisher. Her area of research is Cyber Security, Blockchain, Machine and Deep Learning, and Wireless Sensor Networks.